Yenepoya Institute of Arts, Science, Commerce, and Management

Project – Low Level Design

on

Dark Web Monitoring for Agricultural Assets

Student Name: Mr. Alan Shibu

Industry Mentor: Mr. Shashank

Guided by: Ms. Divya N Shetty

Table of Contents

No.	Section Title	Page
1	**Introduction**	3
1.1	Scope of the Document	3
1.2	Intended Audience	3
1.3	System Overview	4
2	**Detailed System Architecture**	5
2.1	High-Level Architecture	5
2.2	Deployment Architecture	6
3	**Low Level System Design**	7
3.1	Component Design	7
3.2	Class Diagrams	12
3..3	Sequence Diagram	18
3.4	Process Workflows	24
3.5	API Design & Specifications	29
3.6	Database Interaction Design	29
3.7	Data Structures & Schema Details	30
3.8	State and Session Management	32
3.9	Caching Mechanism	33
3.11	Security Implementation	35
3.12	Performance Optimization	37
4	**Interfaces**	39
6	**Testing Strategy**	41
7	**References**	42

1. Introduction

1.1 Scope of the Document

This Low-Level Design (LLD) document defines the detailed implementation blueprint for the Dark Web Monitoring System for Agricultural Assets. Building upon the High-Level Design (HLD), it translates architectural components into concrete, class-level structures and interactions. The document covers module-wise design across all nine system components, including class definitions, relationships, and responsibilities, along with sequence diagrams for critical workflows. It further specifies API contracts, database interaction patterns, and key design considerations such as state management, caching strategies, and structured error handling. Additionally, the document outlines security mechanisms, performance optimization techniques, configuration management practices, and the overall testing approach to ensure a robust and scalable implementation.

The scope of this document is limited to internal system design and implementation details. It includes class-level design for all modules, core workflow sequence diagrams, API request and response specifications, database schema design with indexing strategies, as well as error handling, retry logic, and caching mechanisms. However, it explicitly excludes areas such as hardware procurement specifications, end-user documentation or training materials, legal and compliance certifications, third-party vendor selection, business continuity planning, and detailed UI/UX mockups, as these aspects fall outside the boundaries of low-level system design.

1.2 Intended Audience

This document is intended for the following stakeholders:

- Backend Developers – Implement module logic, APIs, and database operationsDevelopers – to implement and integrate various system components based on the defined architecture
- Frontend Developers – Develop dashboard UI and integrate WebSocket featuresProject Evaluators / Mentors – to assess the feasibility, design quality, and effectiveness of the proposed solution

- Security Analysts – Analyse alert generation and threat detection logic

1.3 System Overview

The system is designed as a modular dark web monitoring platform consisting of nine core modules organized in a pipeline architecture. Each module performs a specific function and passes processed data to the next stage, enabling a structured flow from data collection to alert generation and visualization. The Config Module manages centralized configuration using ConfigurationManager, while the Tor Module ensures anonymous network access through TorManager and TorConnectionValidator. The Crawler Module, built with DarkWebCrawler, CrawlerQueue, and CrawledPage, is responsible for recursively traversing .onion sites and collecting raw data.

The collected data is then processed through multiple analytical layers. The Preprocessor Module handles text cleaning and entity extraction using DocumentProcessor, EntityExtractor, and TextCleaner. The Keyword Detector Module performs exact, fuzzy, and regex-based matching via KeywordDetector, PatternMatcher, and FuzzyMatcher. Further analysis is carried out by the NLP Analyzer Module, which uses NLPAnalyzer, IntentClassifier, and AgricultureNER for intent classification and domain-specific named entity recognition.

Finally, the processed insights are stored, evaluated, and presented to end users. The Storage Module, implemented using DatabaseManager and DataStorageService, manages data persistence in SQLite or PostgreSQL. The Alert Module generates and escalates alerts based on severity using AlertGenerator, AlertChannelManager, and EscalationPolicy. The Dashboard Module provides a web-based interface with real-time updates, built using a Flask application, SocketIO for live communication, and Plotly charts for data visualization.

Work Flow:

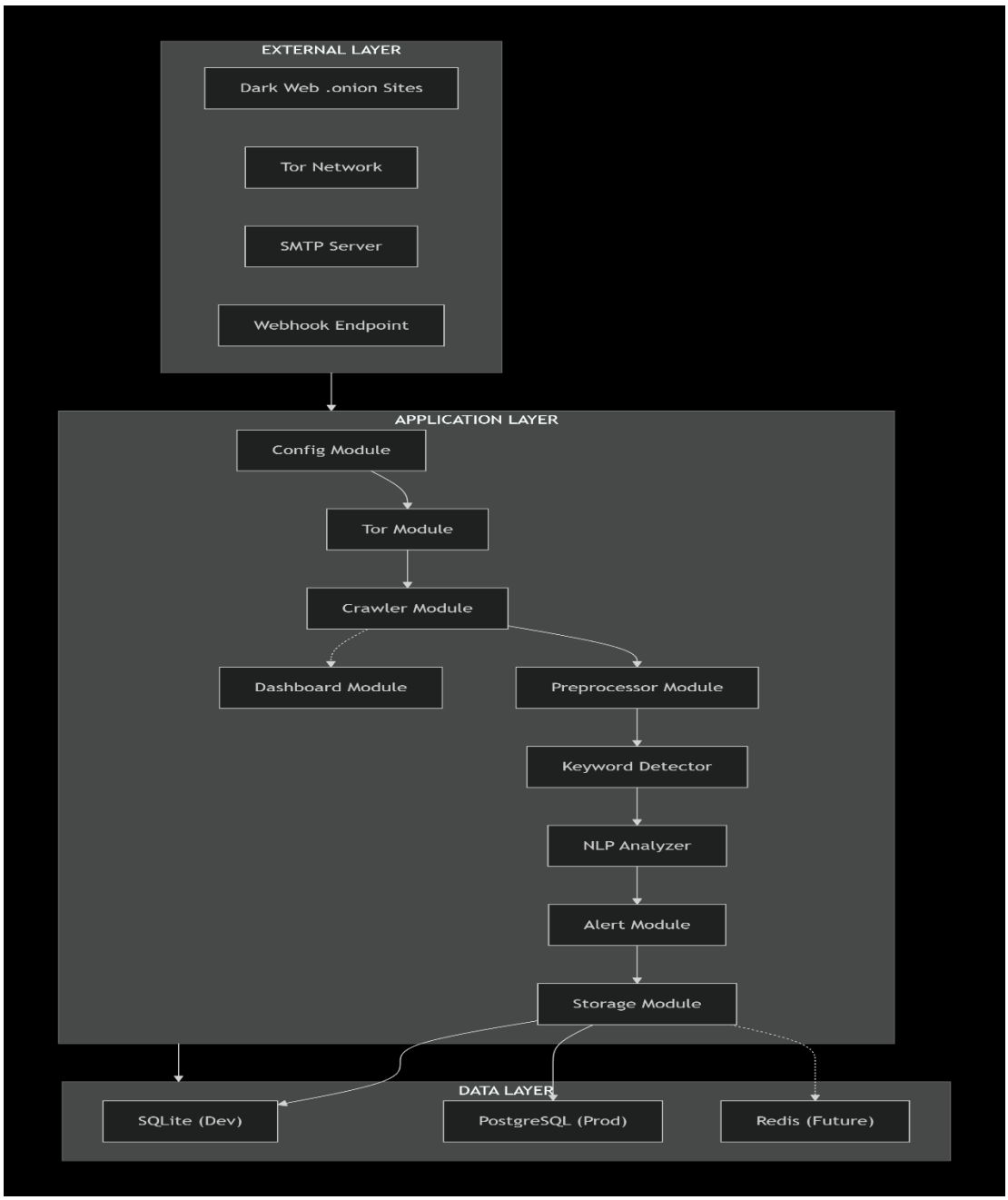
Seed URLs → Tor → Crawler → Preprocessor → Keyword Detection → NLP Analysis → Generation → Storage → Dashboard

2. Detailed System Architecture

2.1 High-Level Architecture

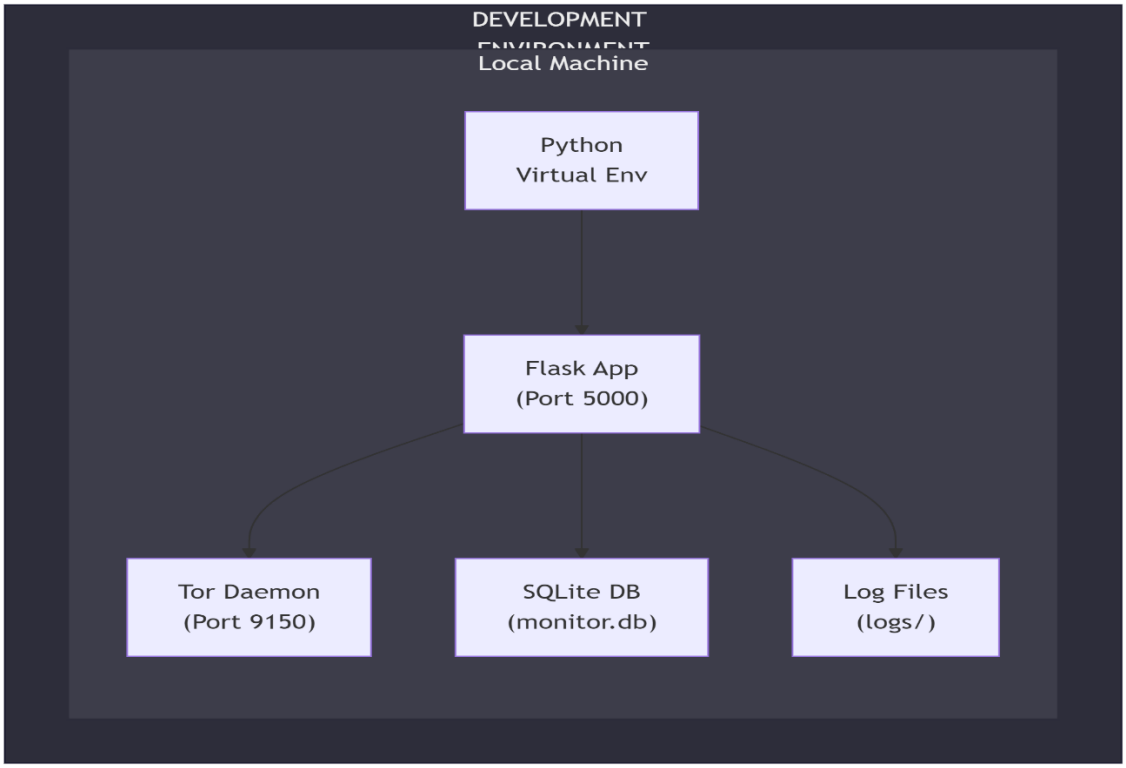
The system follows a modular pipeline architecture with nine loosely coupled modules. Each module has a single responsibility and communication via well-defined data structures.

Architecture Diagram:



Module	Depends On	Depended Upon By
Config	None	All modules
Tor	Config	Crawler
Crawler	Config, Tor	Preprocessor
Preprocessor	Config	Keyword Detector, NLP Analyzer
Keyword Detector	Config, Preprocessor	Alert, Storage
NLP Analyzer	Config, Preprocessor	Alert, Storage
Alert	Config, Keyword Detector, NLP Analyzer, Storage	Dashboard
Storage	Config	Crawler, Preprocessor, Keyword Detector, NLP Analyzer, Alert, Dashboard
Dashboard	Config, Storage	None (User-facing)

2.2 Deployment Architecture



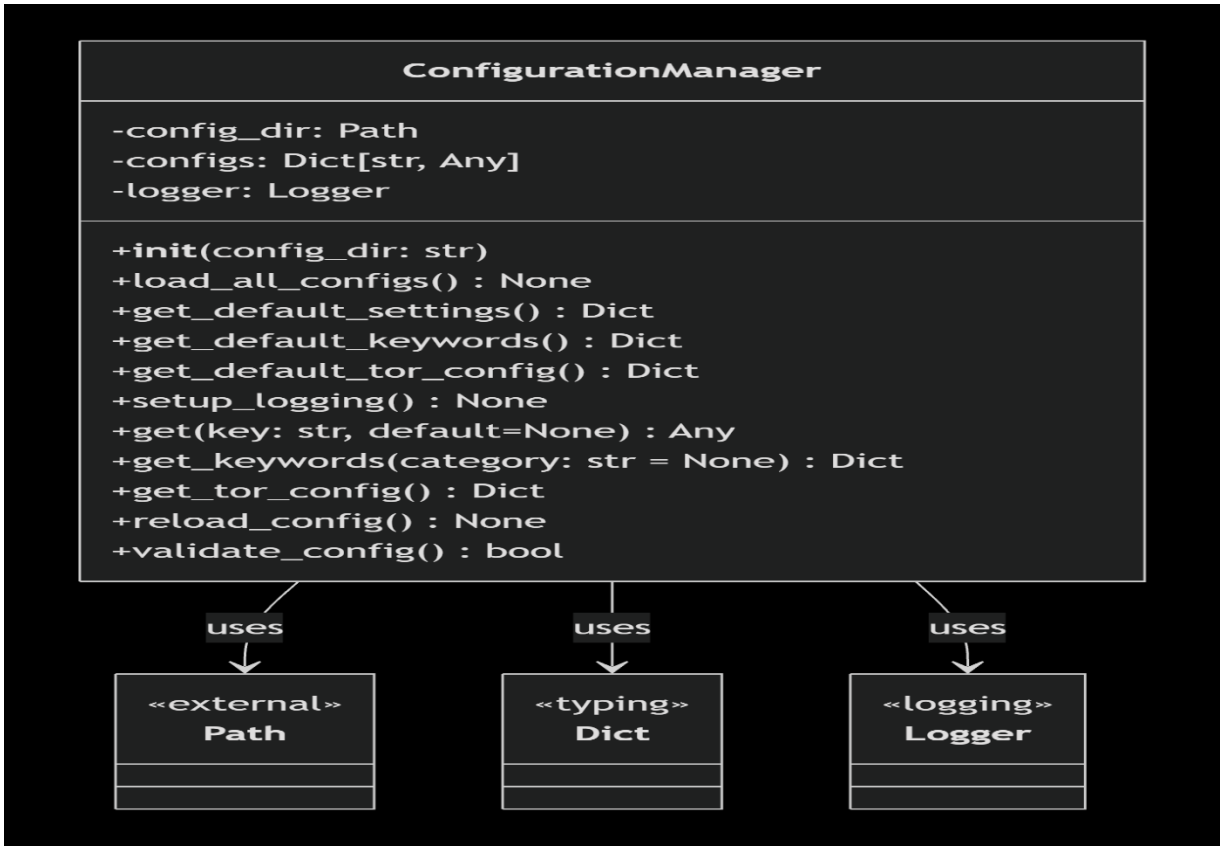
3. Low Level System Design

3.1 Component Design

This section provide detailed breakdown of each of the nine modules, including their responsibilities, key classes, methods, and internal data flow.

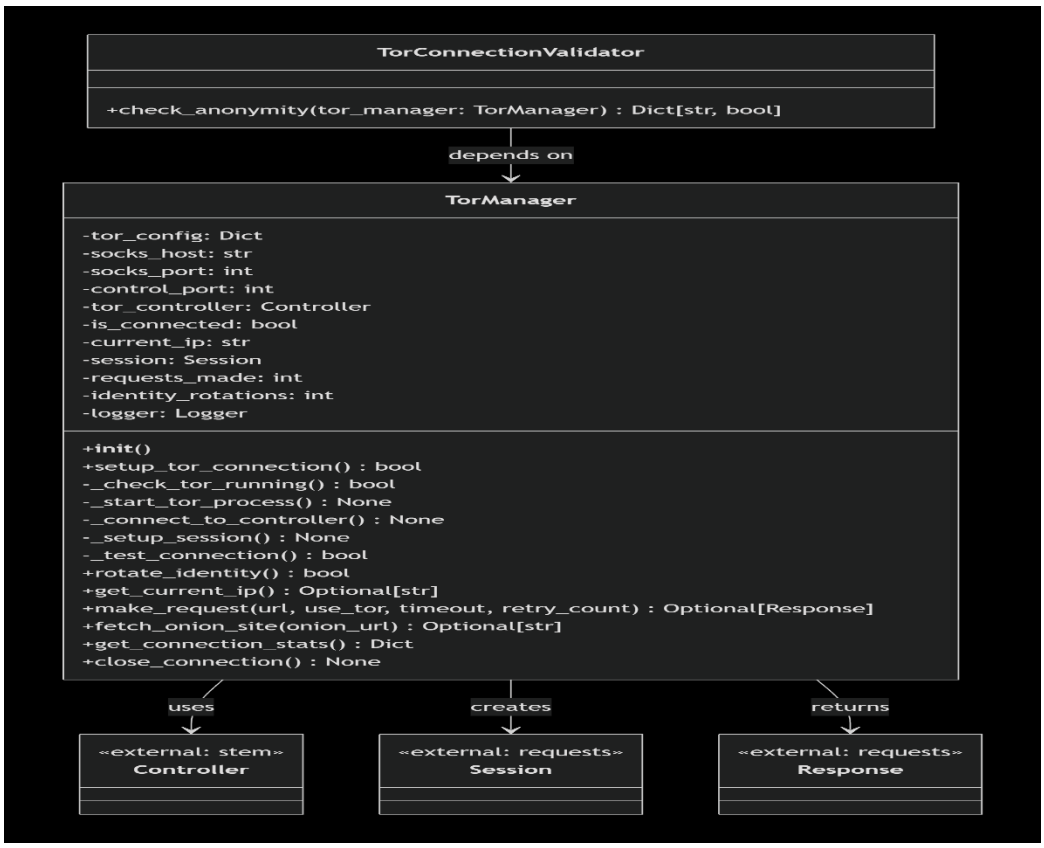
3.1.1 Module 1: Config Module

Centralized configuration management for the entire application



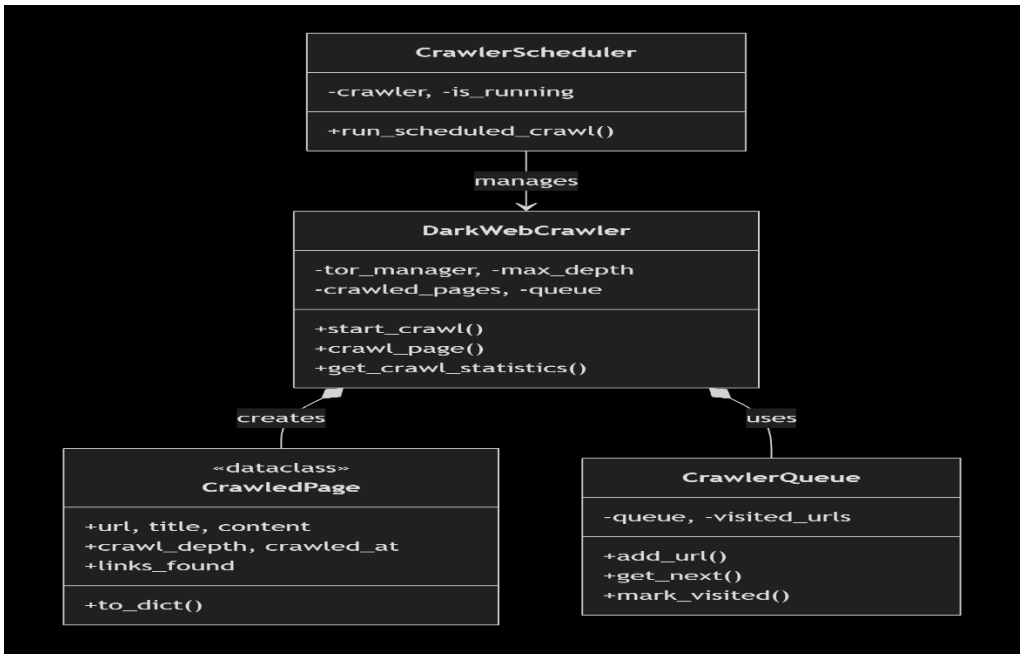
3.1.2 Module 2: Tor Module

Provide anonymous network connectivity via Tor, handles identity rotation, and manages secure requests to .onion sites



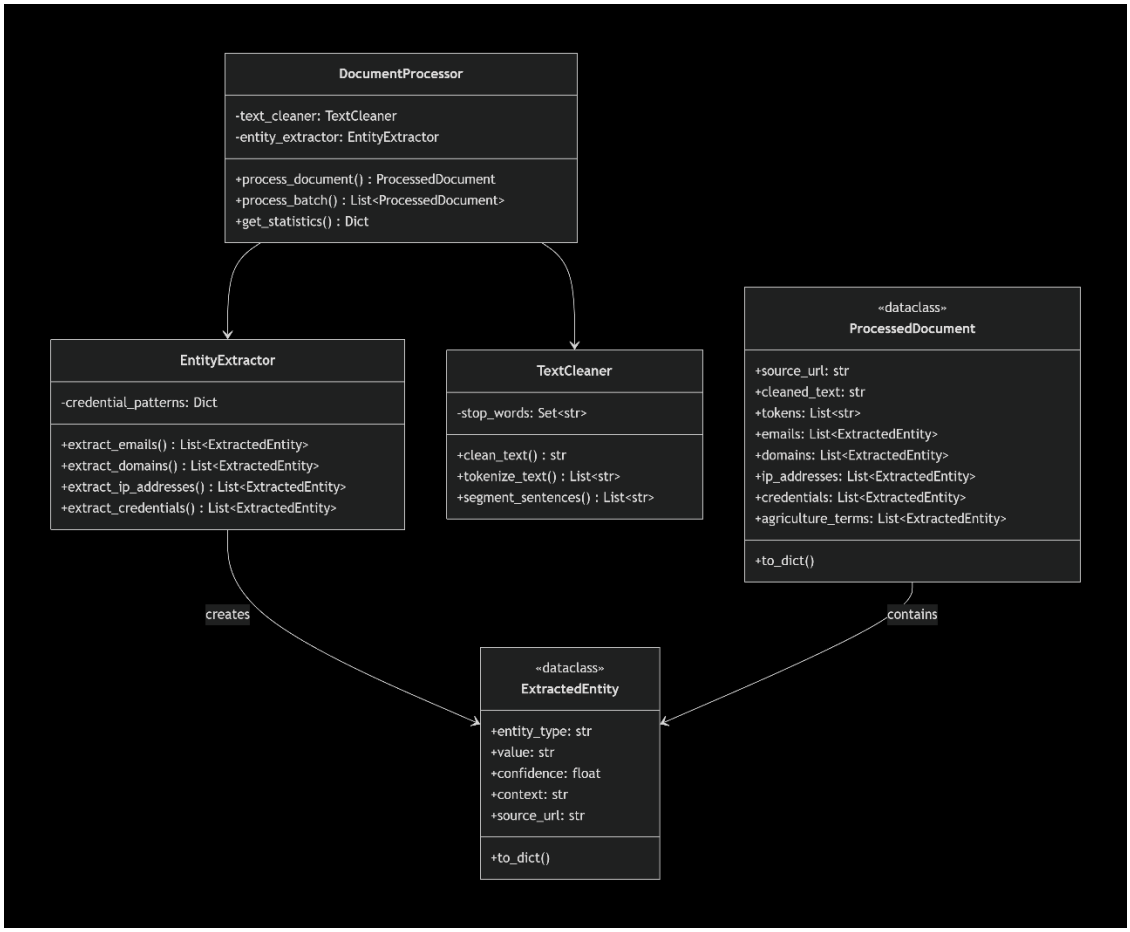
3.1.3 Module 3: Crawler Module

Recursively crawls .onion sites, extracts links, respects depth limits, and manages crawl queue with priority scoring.



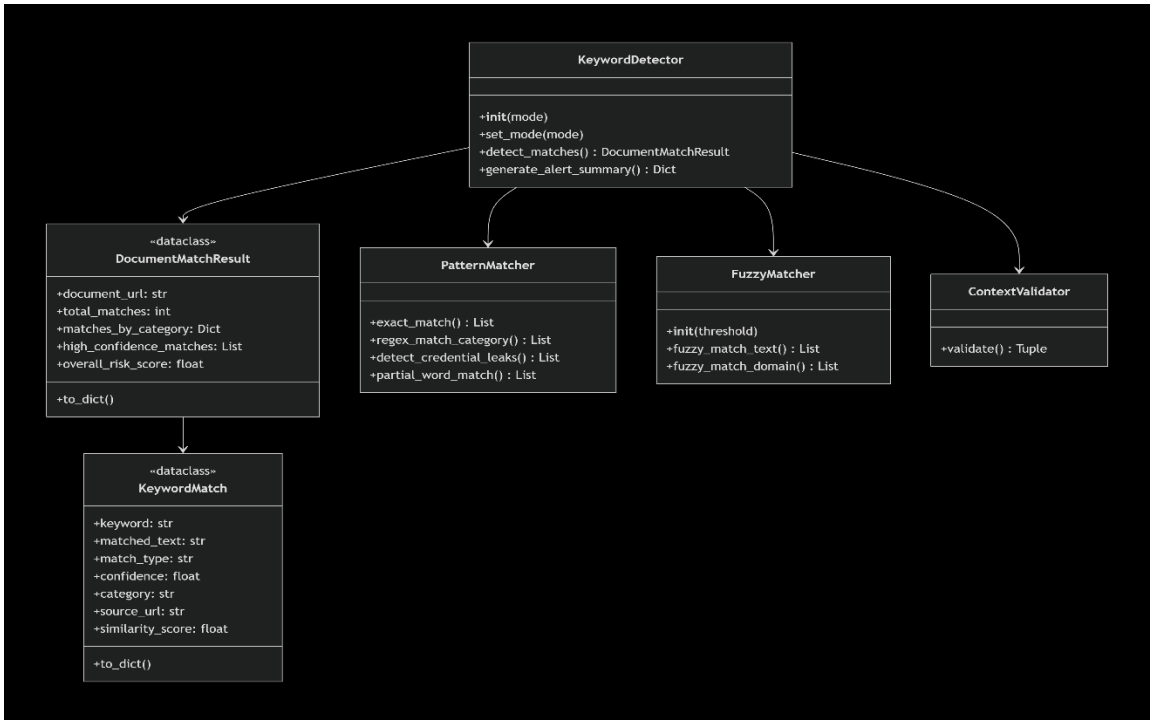
3.1.4 Module 4: Preprocessor Module

Cleans raw HTML/text, normalize content, and extracts entities.



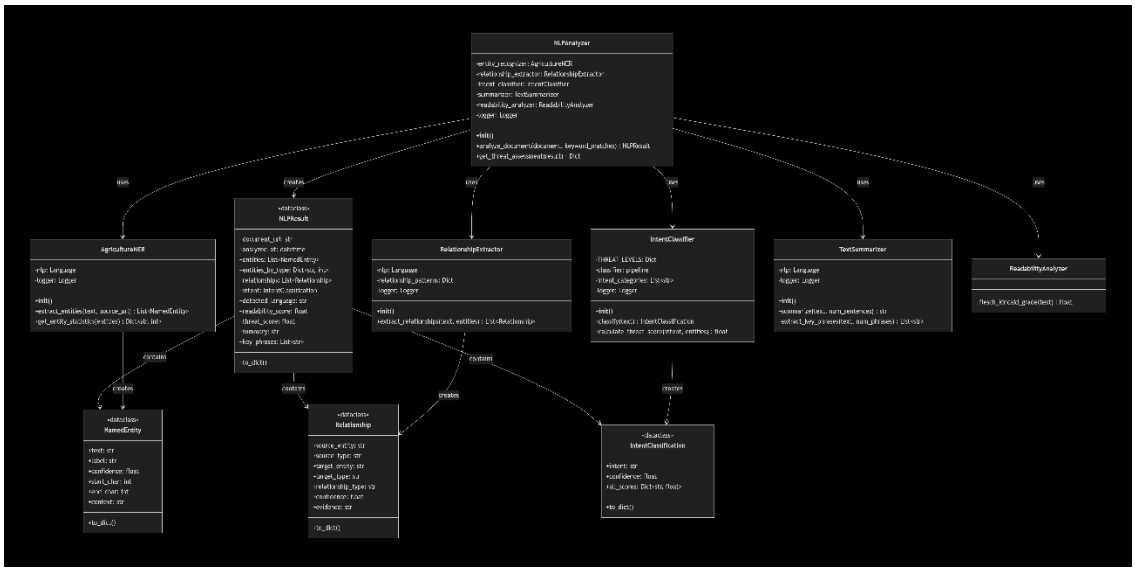
3.1.5 Module 5: Keyword Detector Module

Detects keywords using exact, fuzzy, and regex pattern matching with context validation



3.1.6 Module 6: NLP Analyzer Module

Performance advanced NLP processing including named entity recognition, intent classification, relationship extraction, and text summarization

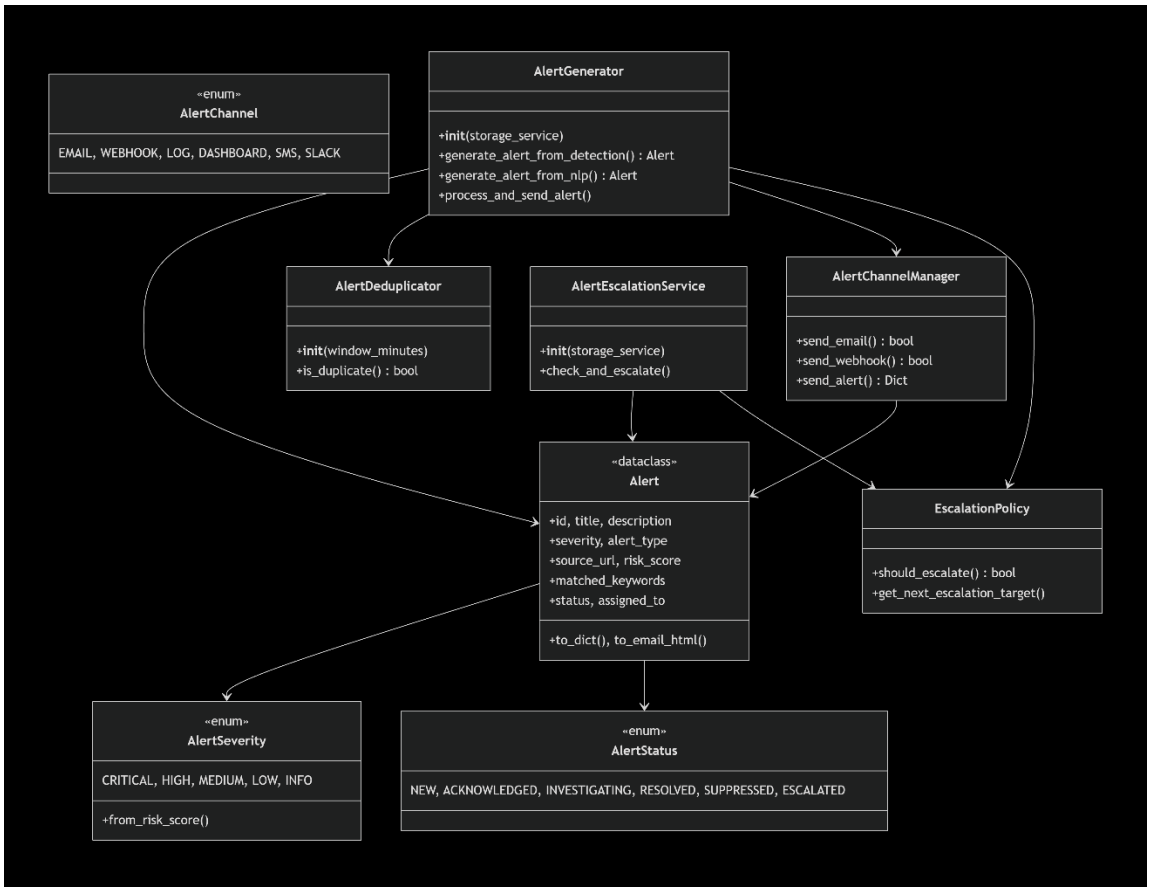


3.1.7 Module 7: Storage Module

Manages database connections, ORM models, and data persistence operation

3.1.8 Module 8: Alert Module

Generates alerts based on detection results, manages severity classification, multi-channel delivery, and escalation policies.



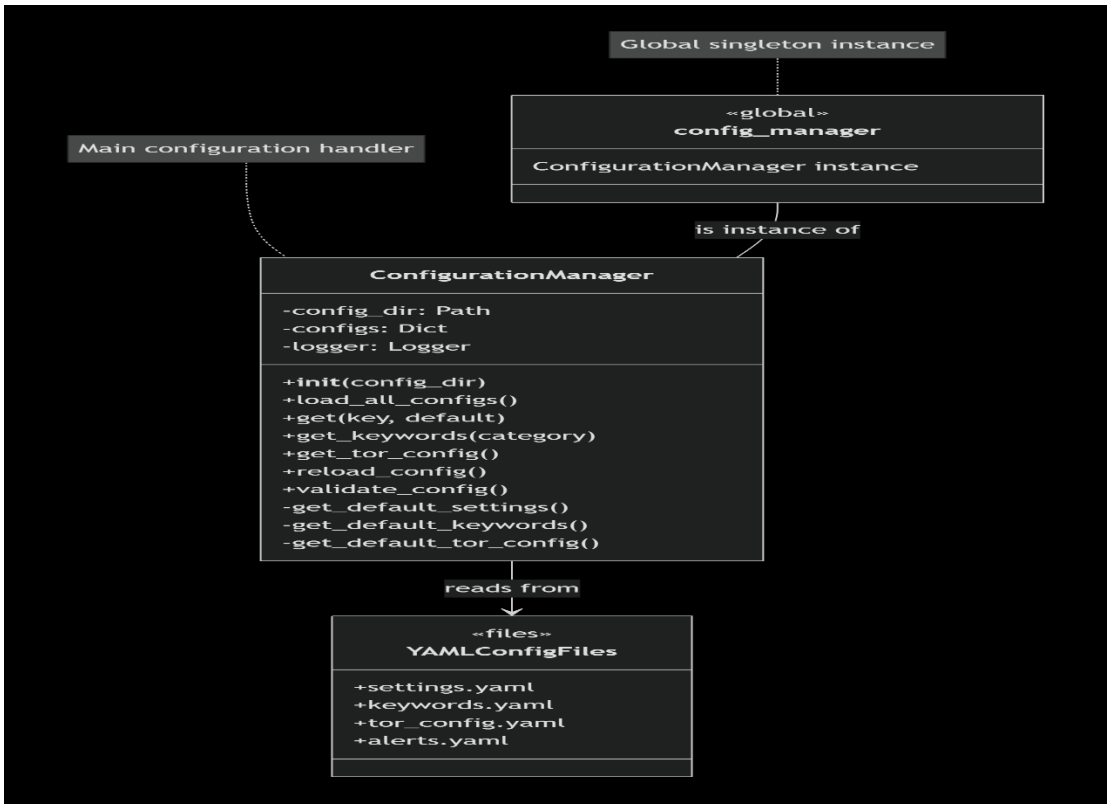
3.1.9 Module 9: Dashboard Module

Provide web-based user interface for monitoring alerts, viewing statistics, and managing system state.

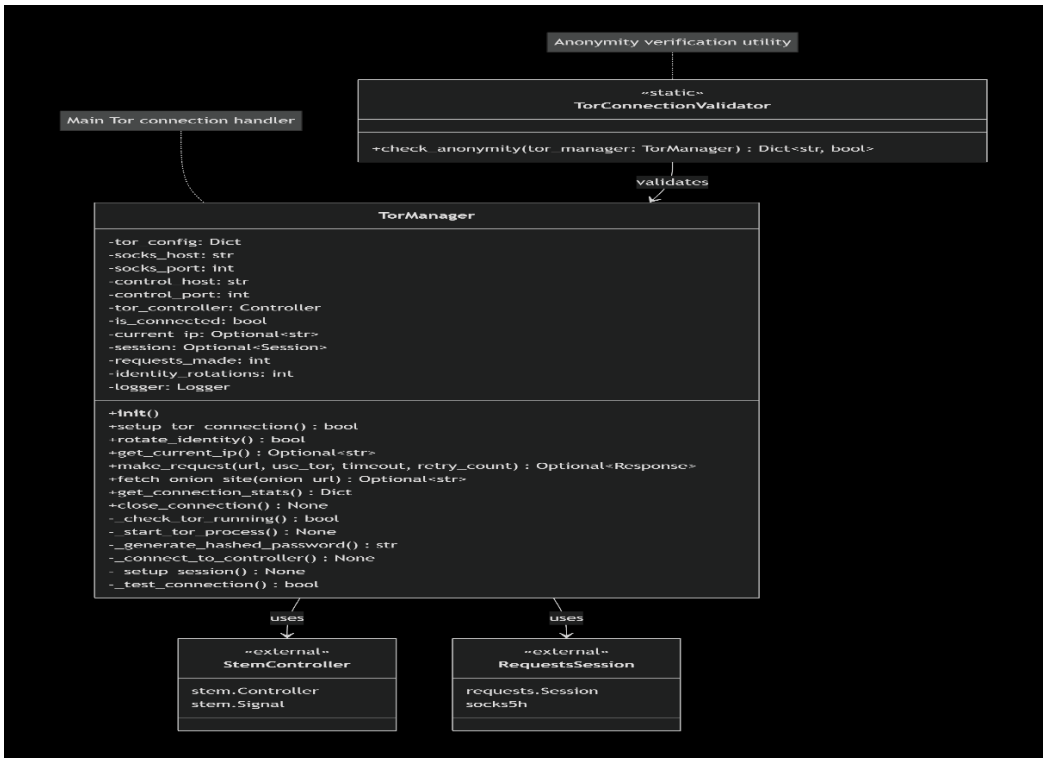
3.2 Class Diagrams

This section provides detailed class diagrams for each module, showing class relationships, inheritance, composition, and method signature

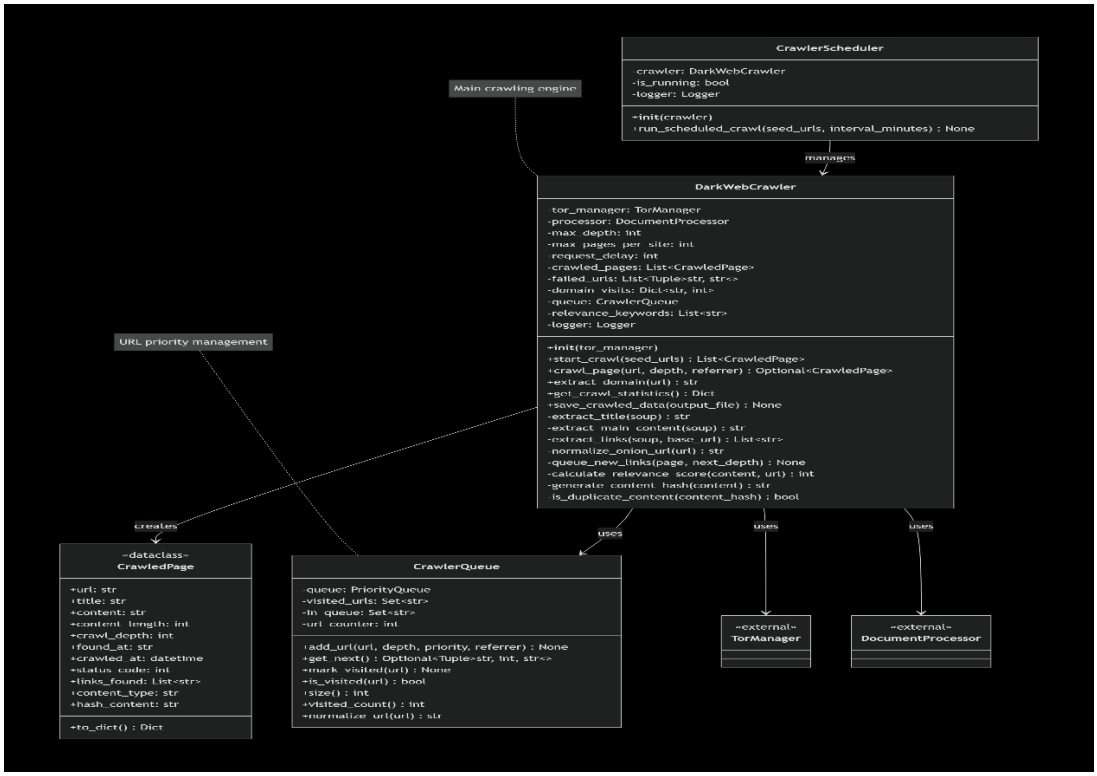
3.2.1 Config Module



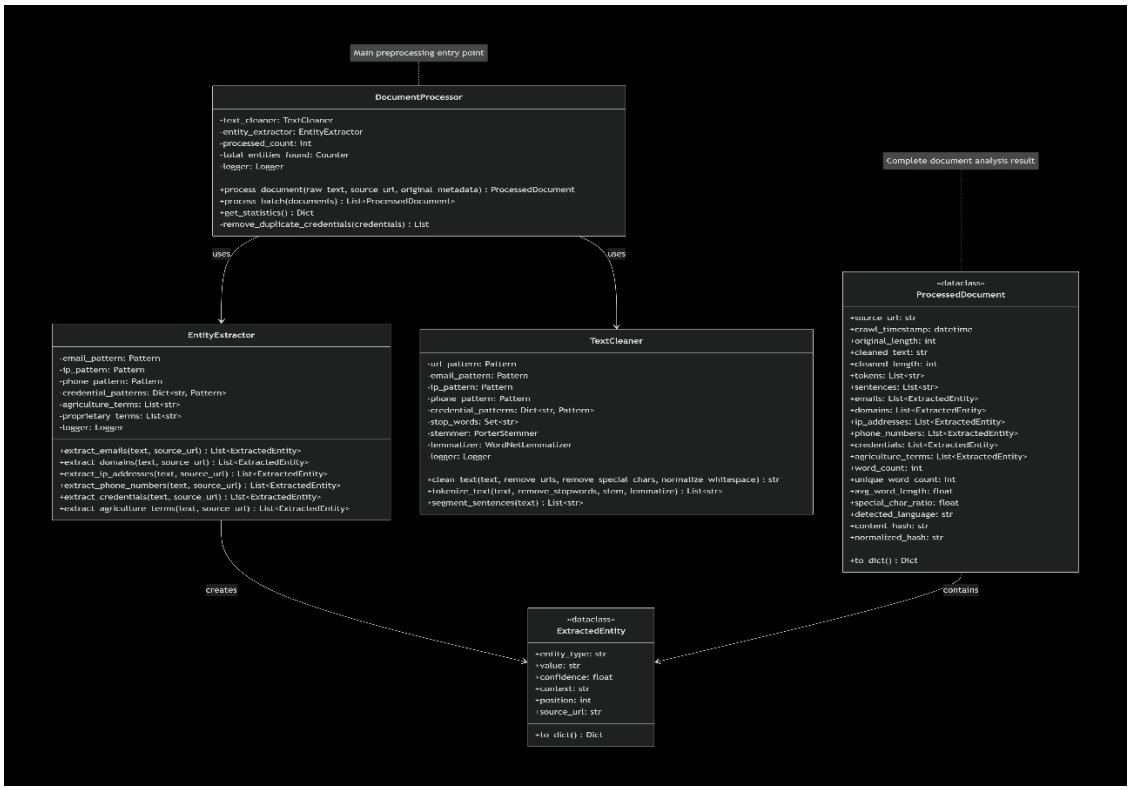
3.2.2 Tor Module



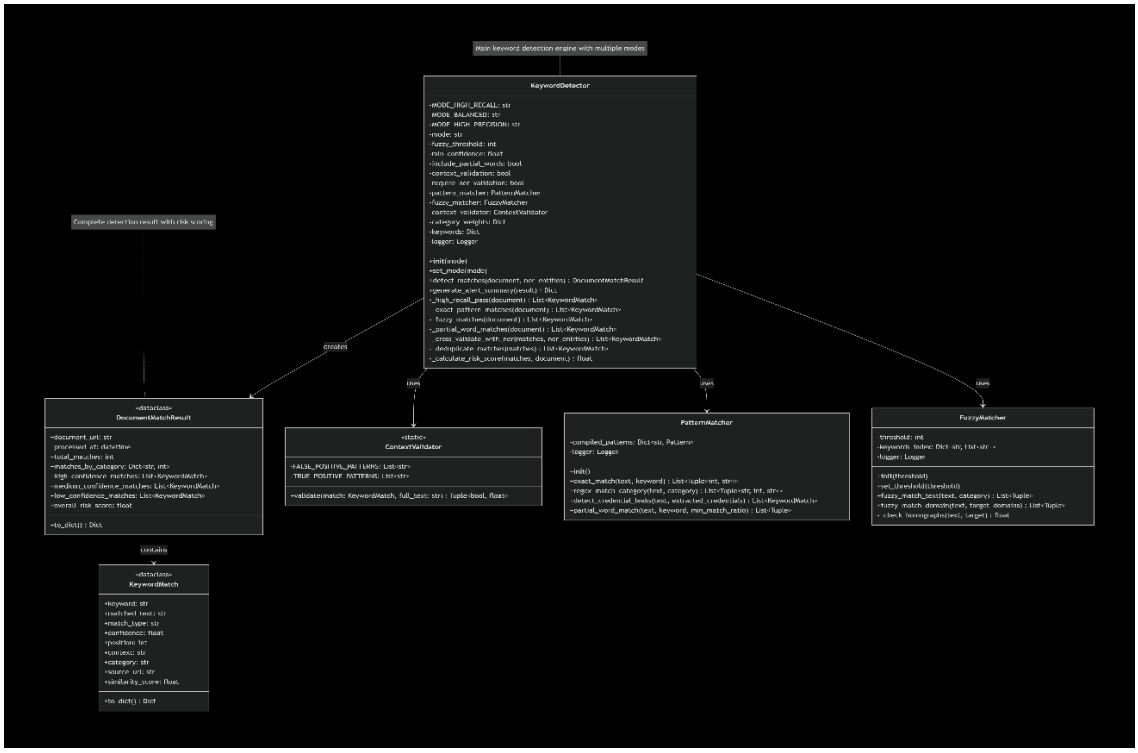
3.2.3 Crawler Module



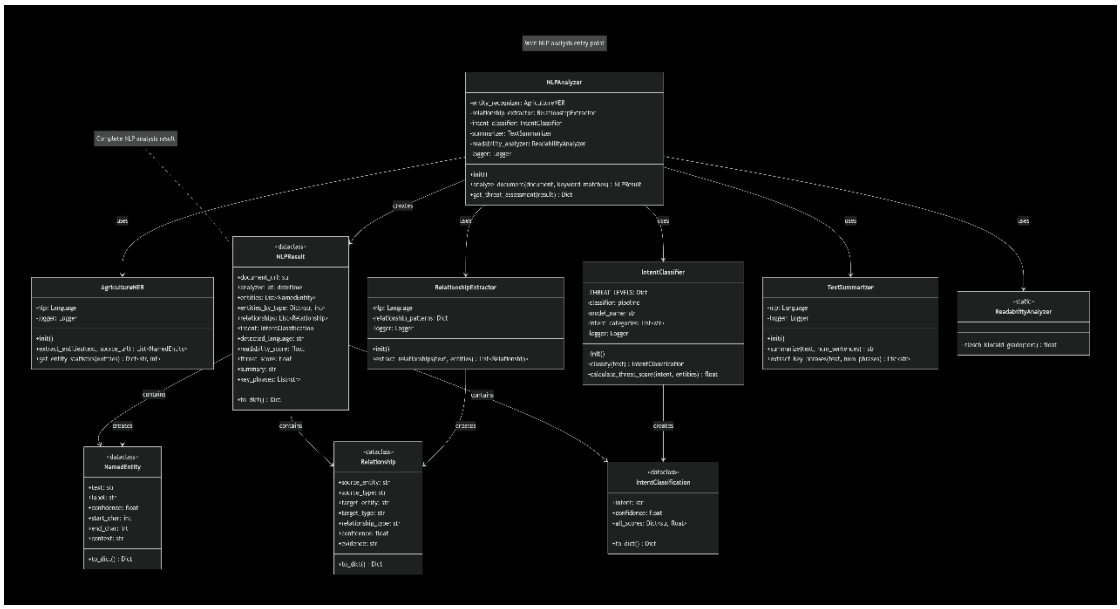
3.2.4 Preprocessor Module



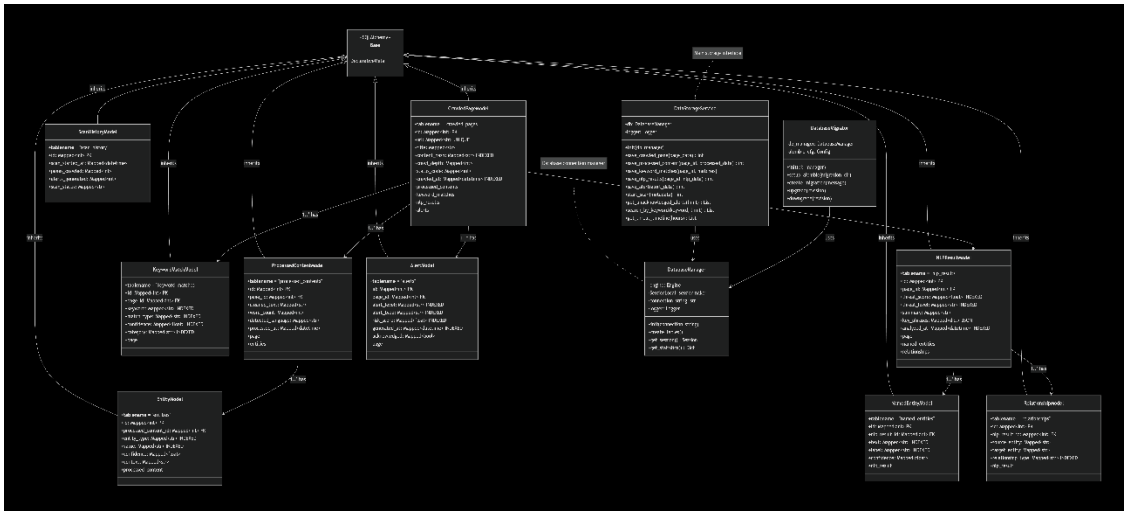
3.2.5 Keyword Detector Module



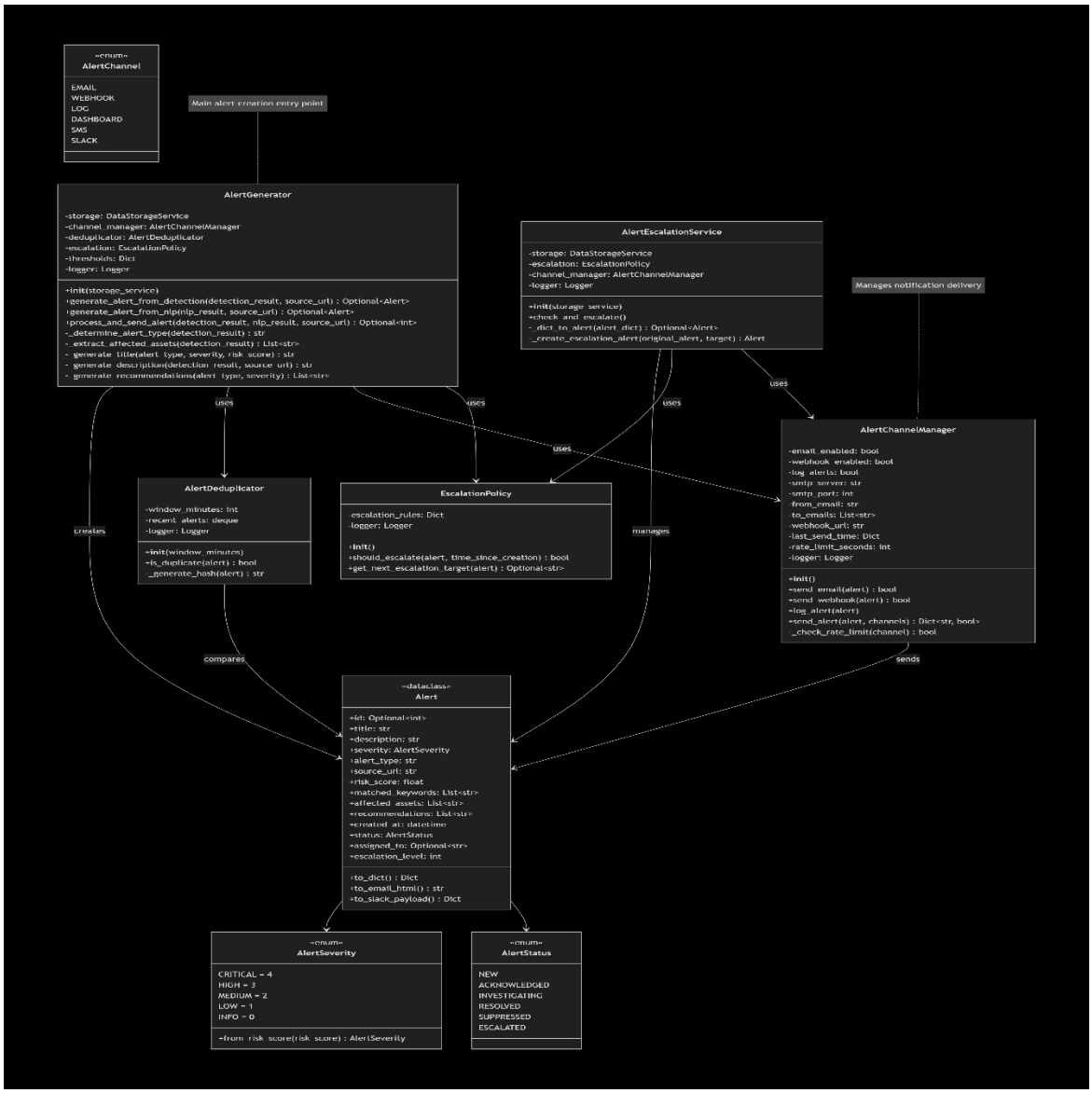
3.2.6 NLP Analyzer Module



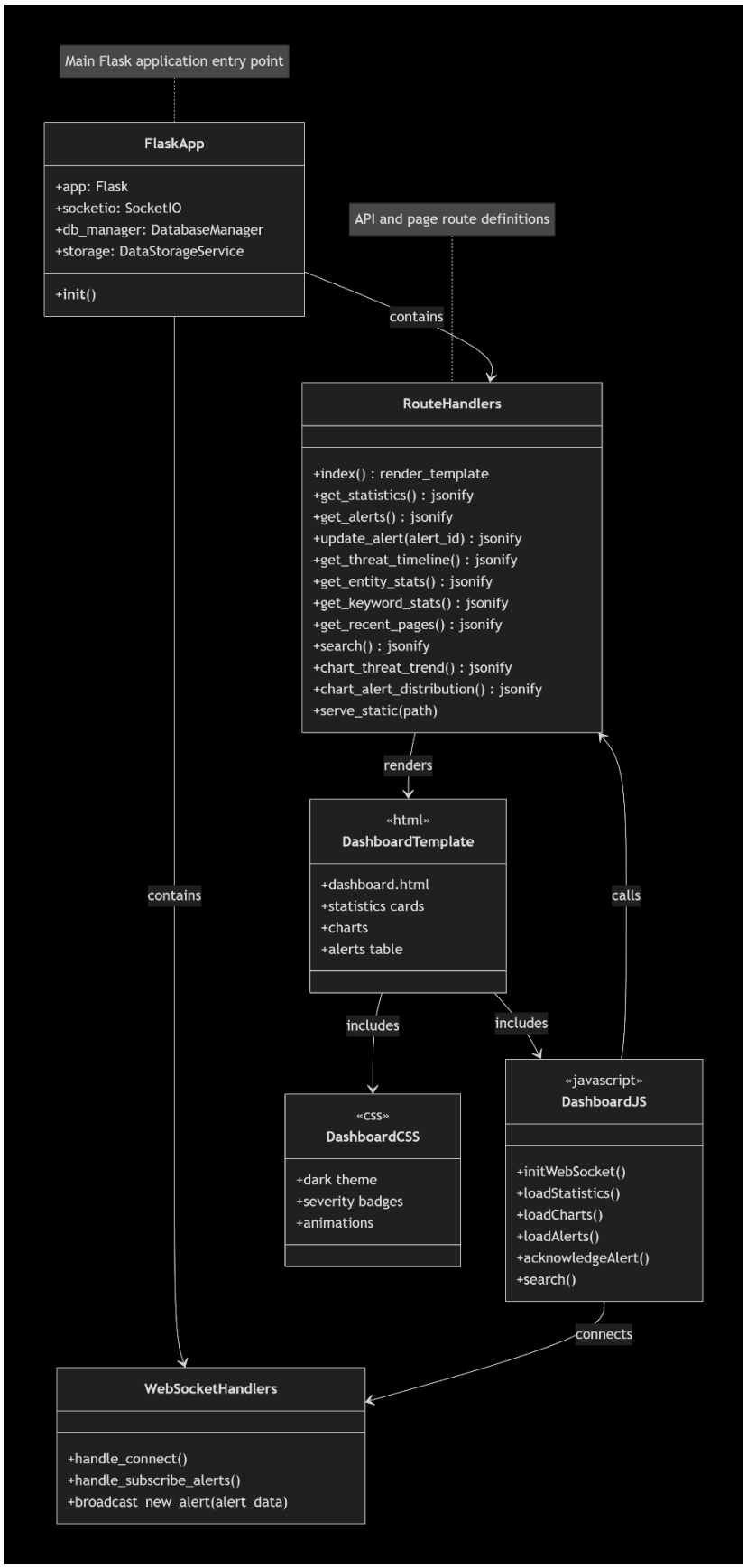
3.2.7 Storage Module



3.2.8 Alert Module



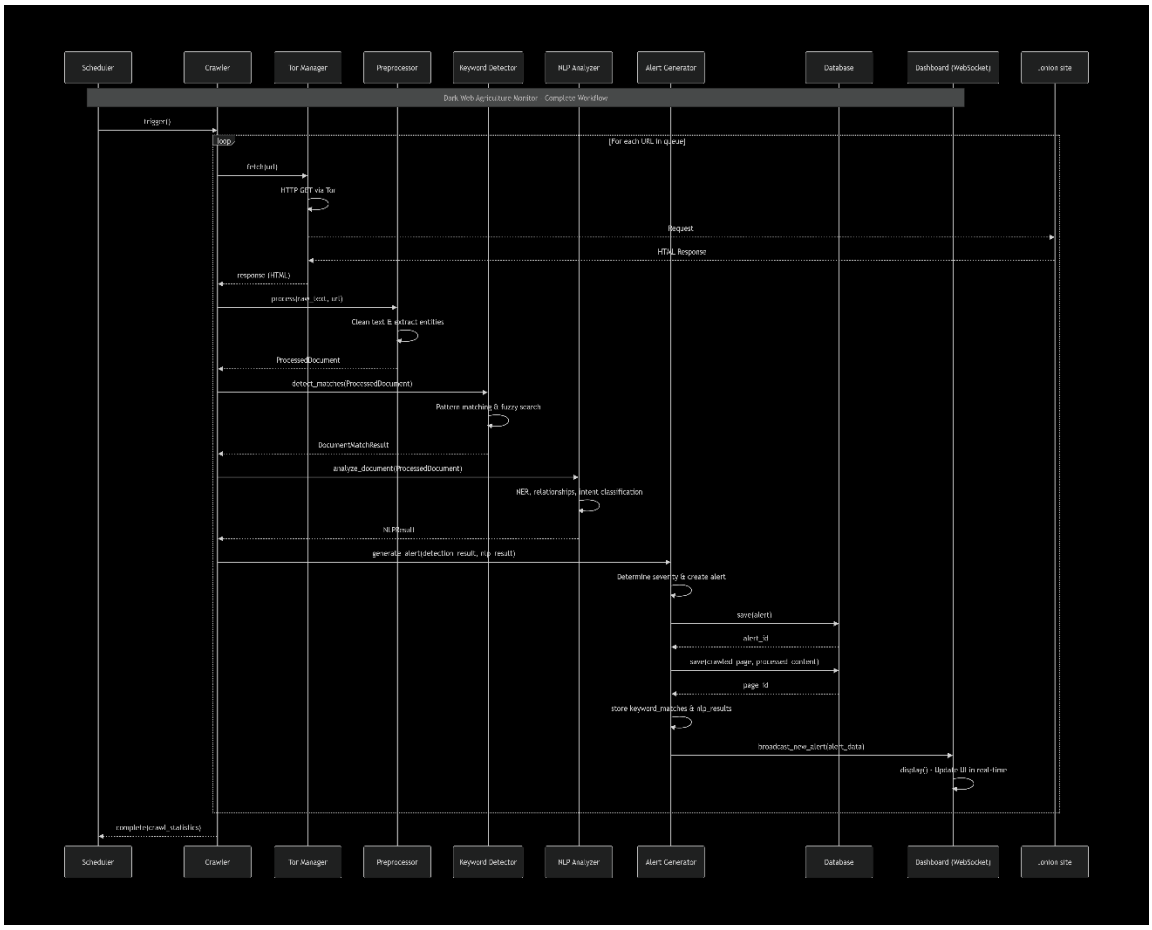
3.2.9 Dashboard Module


Main Flask application entry point
FlaskApp
+app: Flask
+socketio: SocketIO
+db_manager: DatabaseManager
+storage: DataStorageService
+init()
API and page route definitions
contains
RouteHandlers
+index() : render_template
+get_statistics() : jsonify
+get_alerts() : jsonify
+update_alert(alert_id) : jsonify
+get_threat_timeline() : jsonify
+get_entity_stats() : jsonify
+get_keyword_stats() : jsonify
+get_recent_pages() : jsonify
+search() : jsonify
+chart_threat_trend() : jsonify
+chart_alert_distribution() : jsonify
+serve_static(path)
renders
«html»
DashboardTemplate
+dashboard.html
+statistics cards
+charts
+alerts table
contains
calls
includes
includes
«CSS»
DashboardCSS
+dark theme
+severity badges
+animations
«javascript»
DashboardJS
+initWebSocket()
+loadStatistics()
+loadCharts()
+loadAlerts()
+acknowledgeAlert()
+search()
connects
WebSocketHandlers
+handle_connect()
+handle_subscribe_alerts()
+broadcast_new_alert(alert_data)

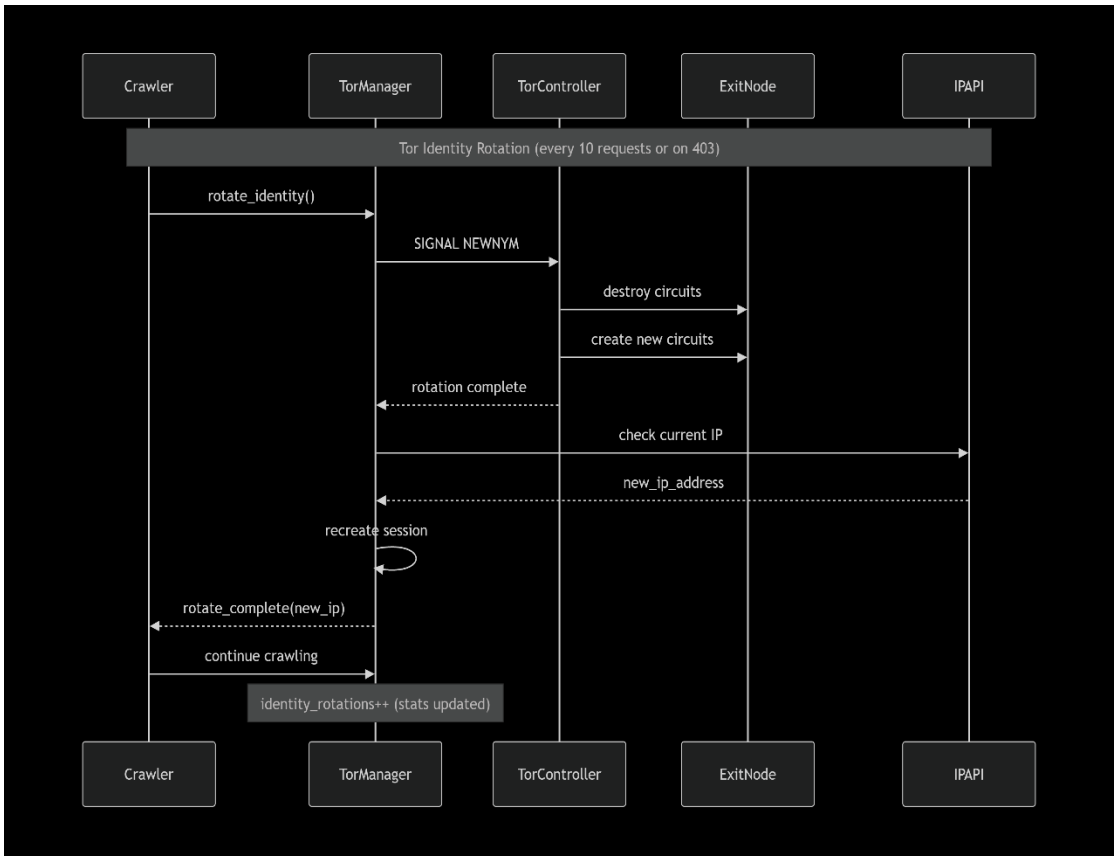
3.3 Sequence Diagrams

This section provides detailed sequence diagram for the core workflow of the Dark Web Agriculture Monitor System.

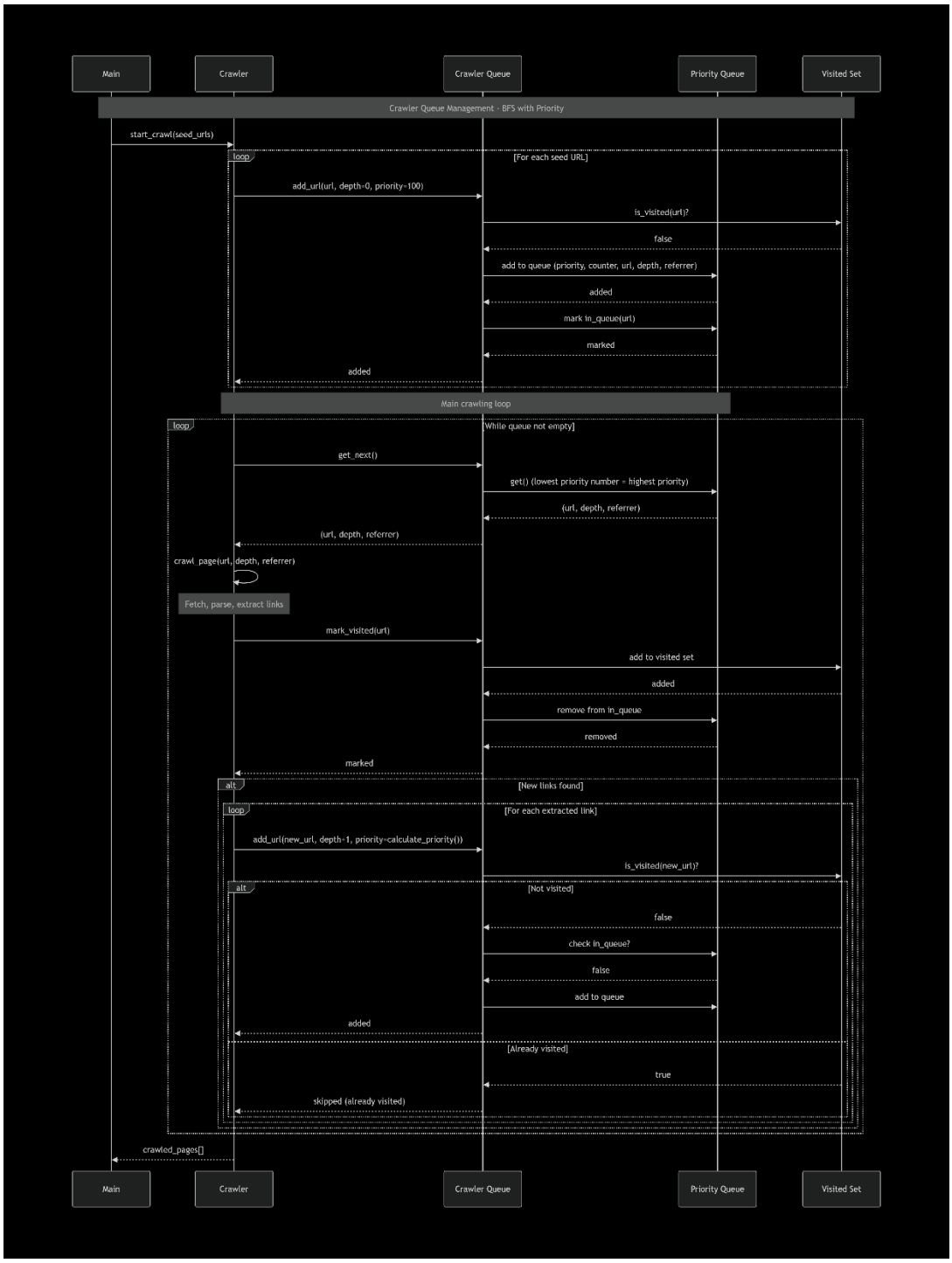
3.3.1 Complete End-to-End



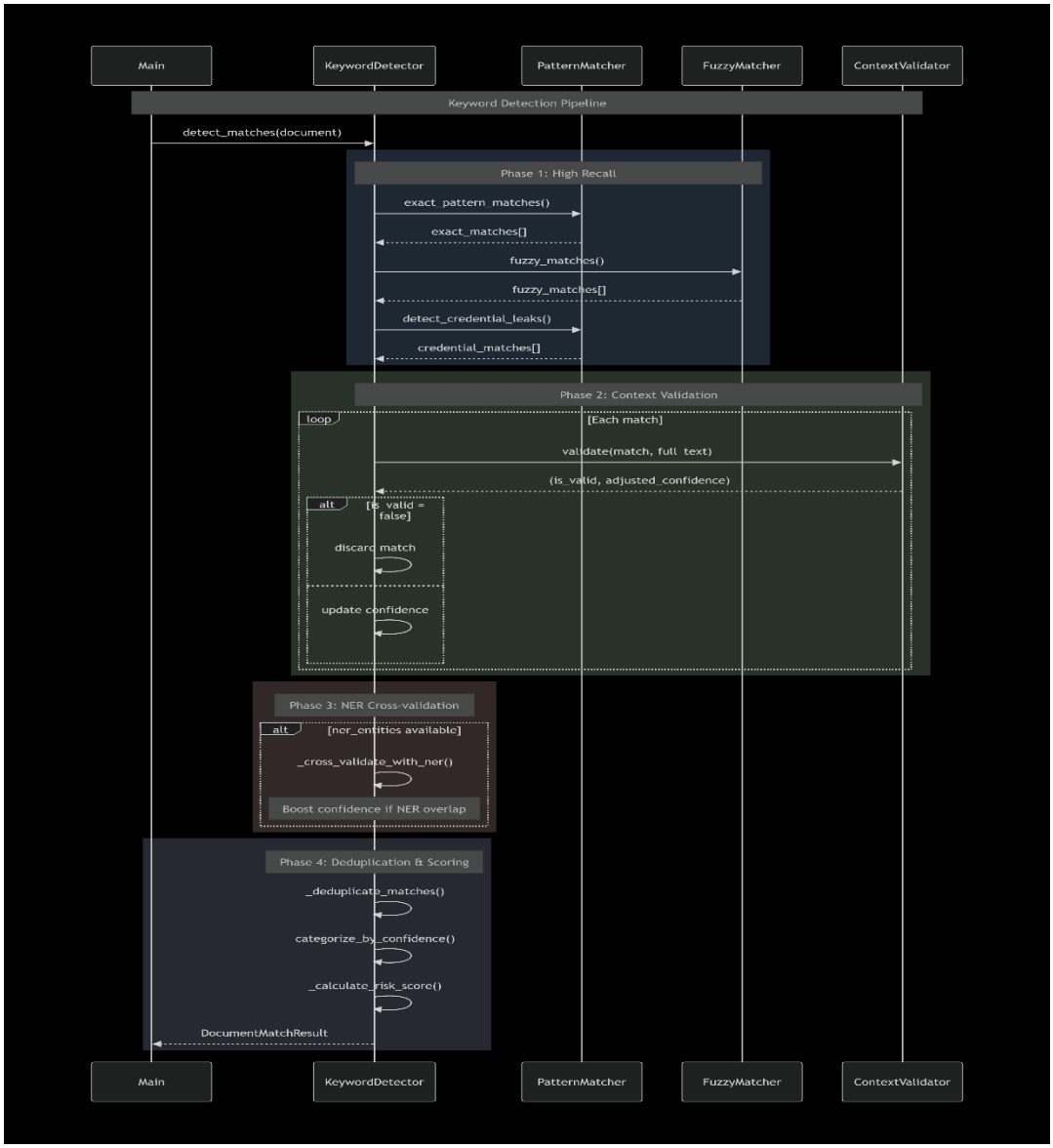
3.3.2 Tor identity Rotation Sequence



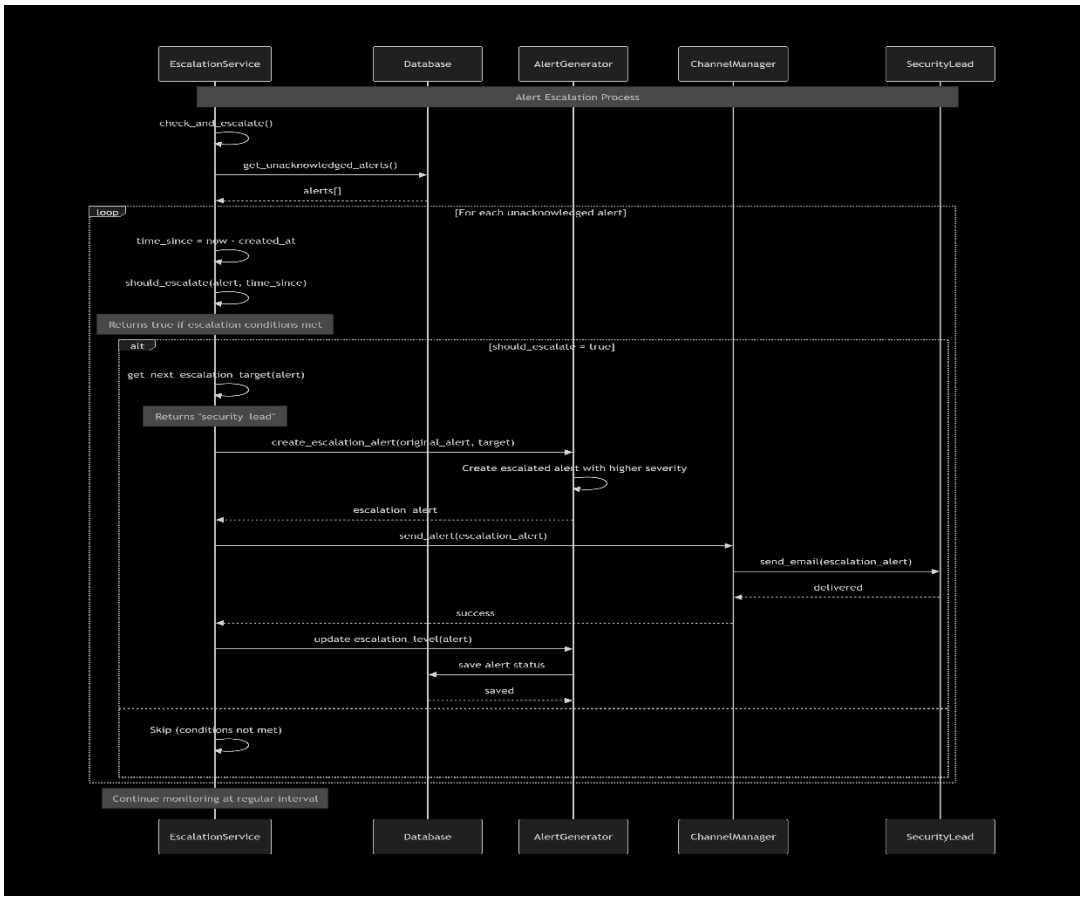
3.3.3 Crawler Queue Management Sequence


Main
Crawler
Crawler Queue
Priority Queue
Visited Set
Crawler Queue Management - BFS with Priority
start_crawl(seed_urls)
loop
[For each seed URL]
add_url(url, depth=0, priority=100)
is_visited(url)?
false
add to queue (priority, counter, url, depth, referrer)
added
mark in_queue(url)
marked
added
Main crawling loop
loop
[While queue not empty]
get_next()
get() (lowest priority number = highest priority)
(url, depth, referrer)
(url, depth, referrer)
crawl_page(url, depth, referrer)
Fetch, parse, extract links
mark_visited(url)
add to visited set
added
remove from in_queue
removed
marked
alt
[New links found]
loop
[For each extracted link]
add_url(new_url, depth+1, priority=calculate_priority())
is_visited(new_url)?
alt
[Not visited]
false
check in_queue?
false
add to queue
added
[Already visited]
true
skipped (already visited)
crawled_pages[]
Main
Crawler
Crawler Queue
Priority Queue
Visited Set

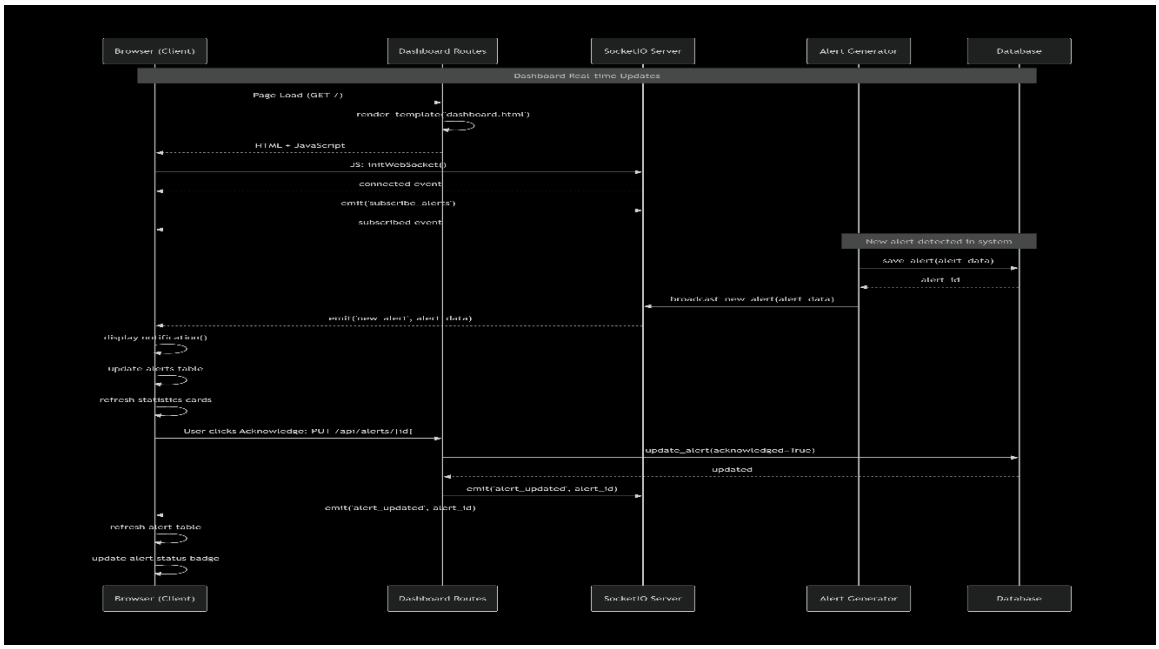
3.3.4 Keyword Detection Two-pass Sequence



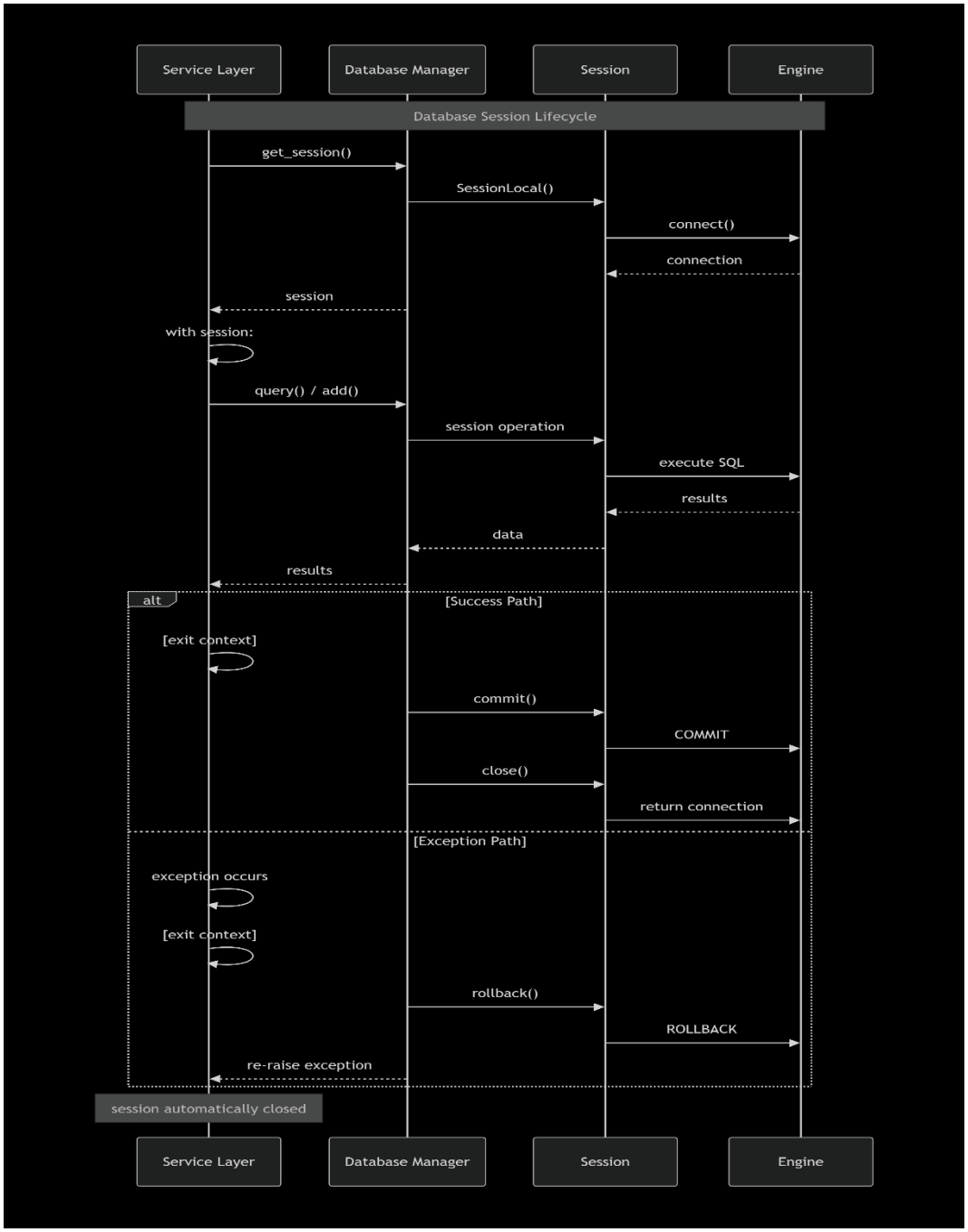
3.3.5 Alert Escalation Sequence



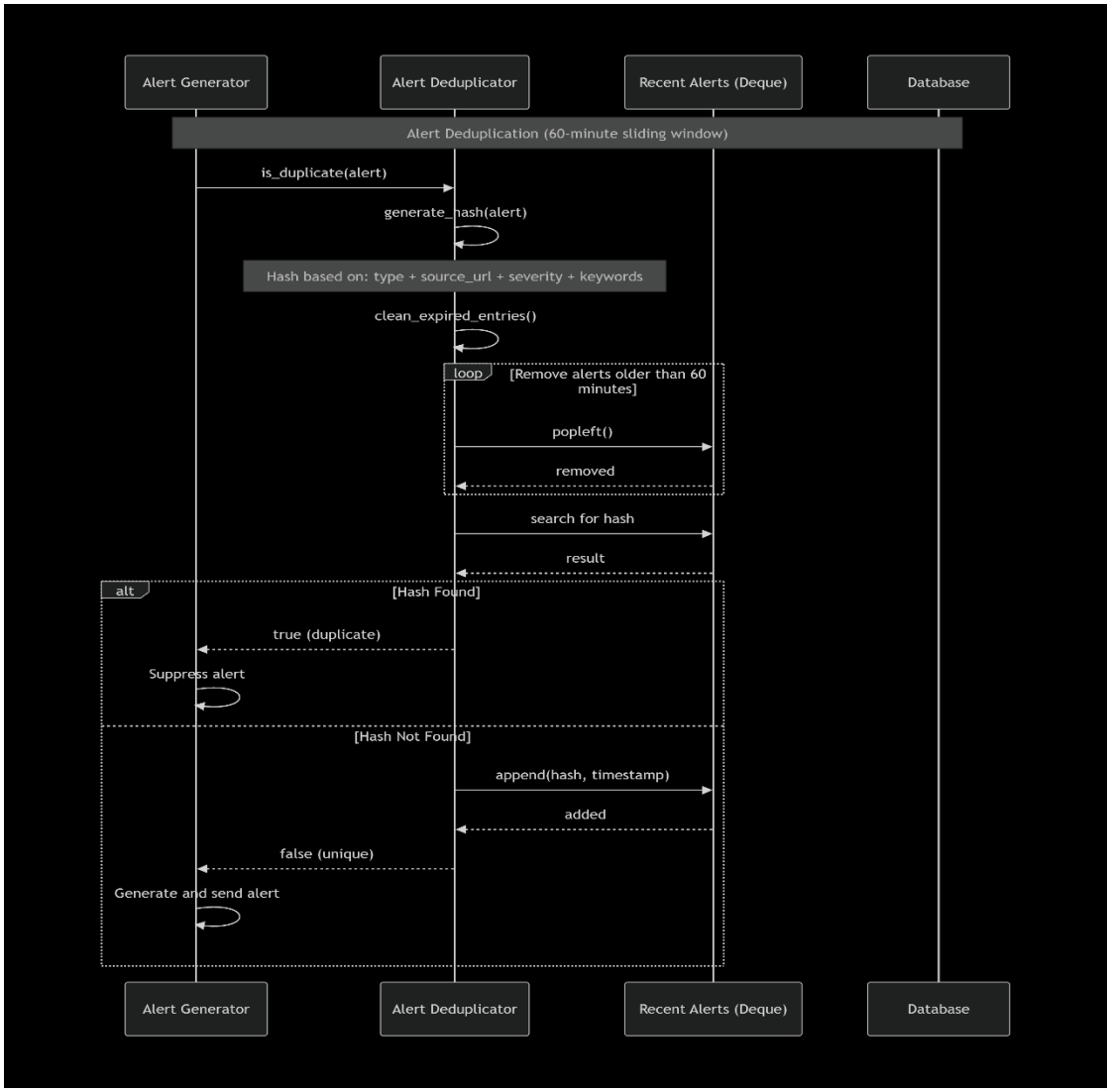
3.3.6 Dashboard Real-Time Update Sequence



3.3.7 Database Session Management Sequence



3.3.8 Alert Deduplication Sequence

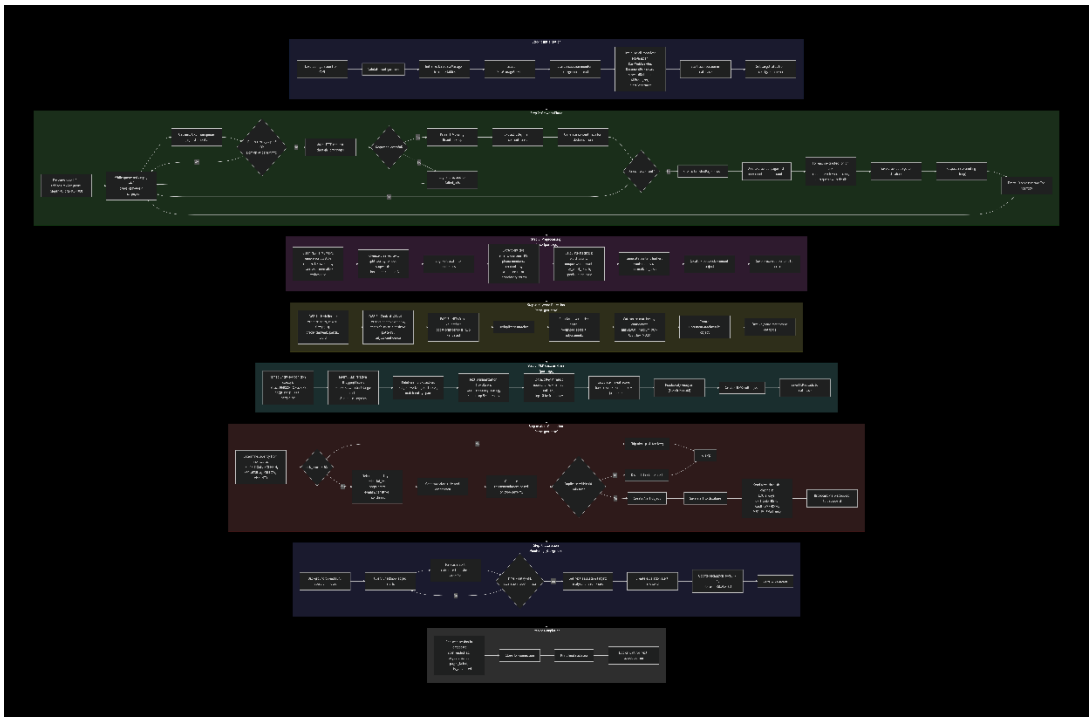


3.4 Process Workflows

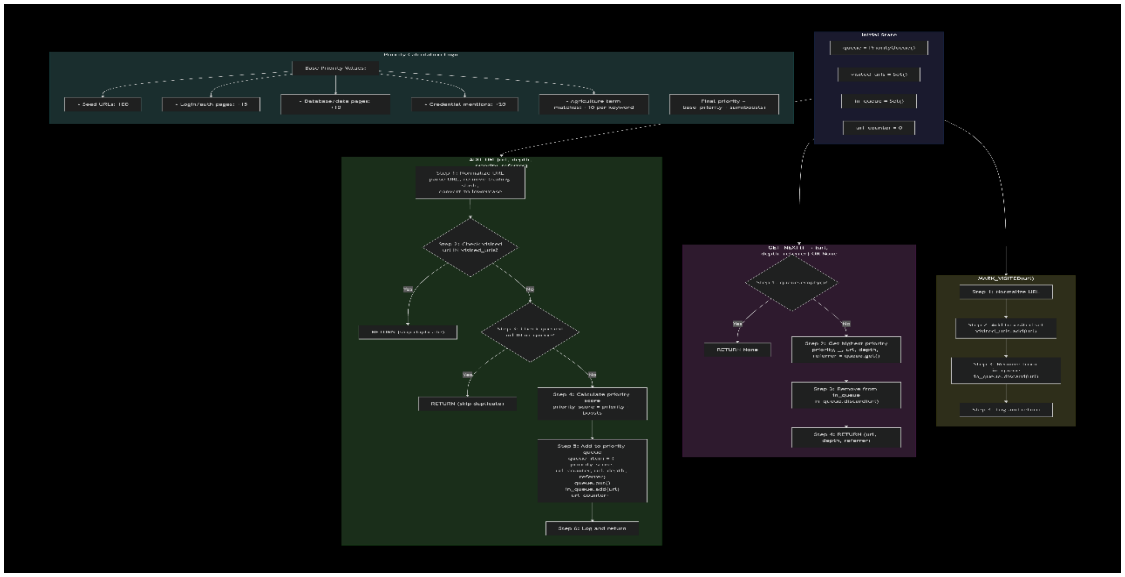
The system is a modular dark web monitoring platform built on a pipeline architecture, where nine core modules work together seamlessly. It begins with the Config Module, which handles centralized configuration through the ConfigurationManager. The Tor Module ensures anonymous network access using components like TorManager and TorConnectionValidator. The Crawler Module then performs recursive traversal of .onion sites using classes such as DarkWebCrawler, CrawlerQueue, and CrawledPage. Finally, the Preprocessor Module cleans and structures the collected data through

DocumentProcessor, EntityExtractor, and TextCleaner, preparing it for further analysis.

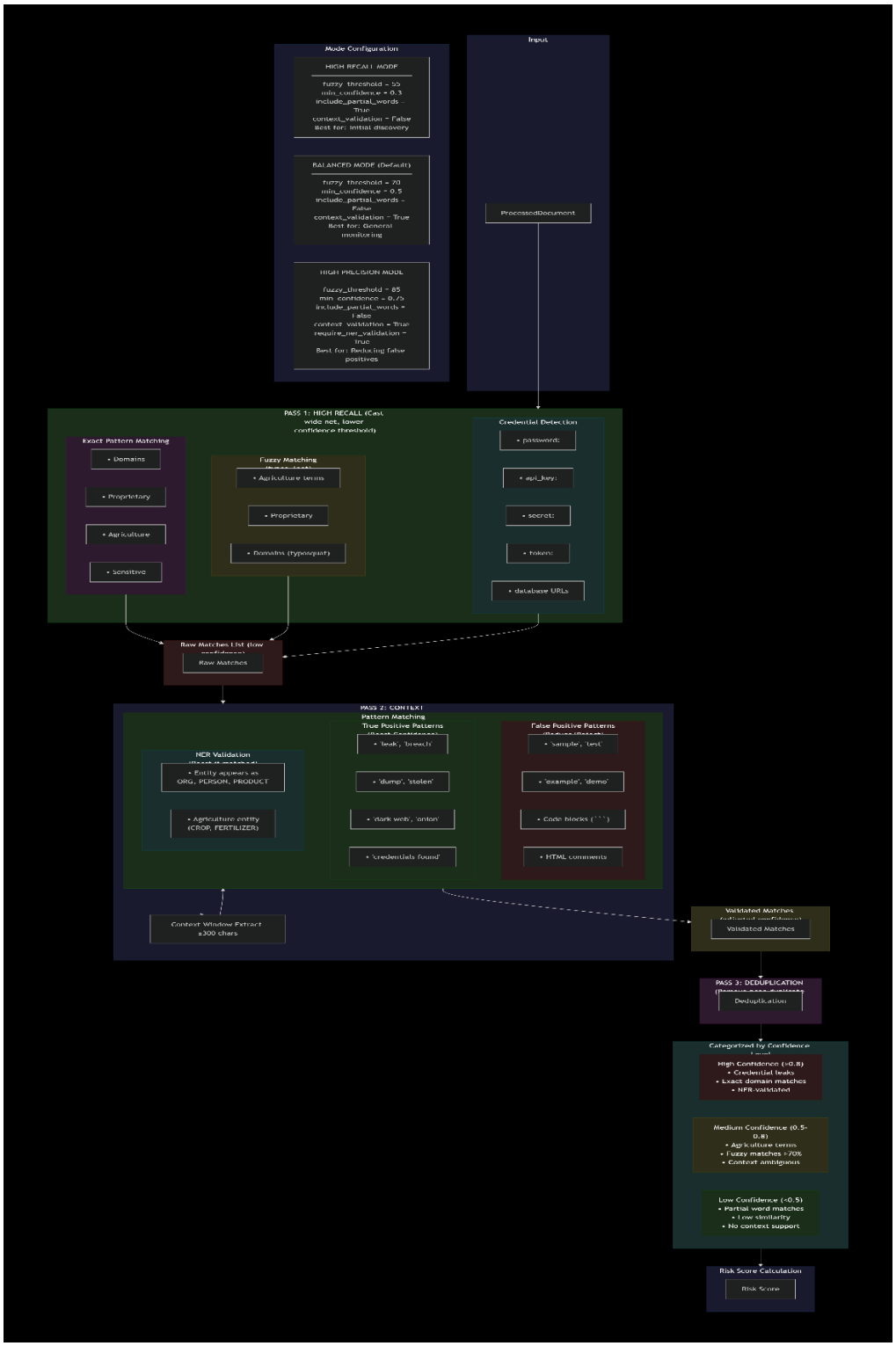
3.4.1 Crawl and Analysis Pipeline



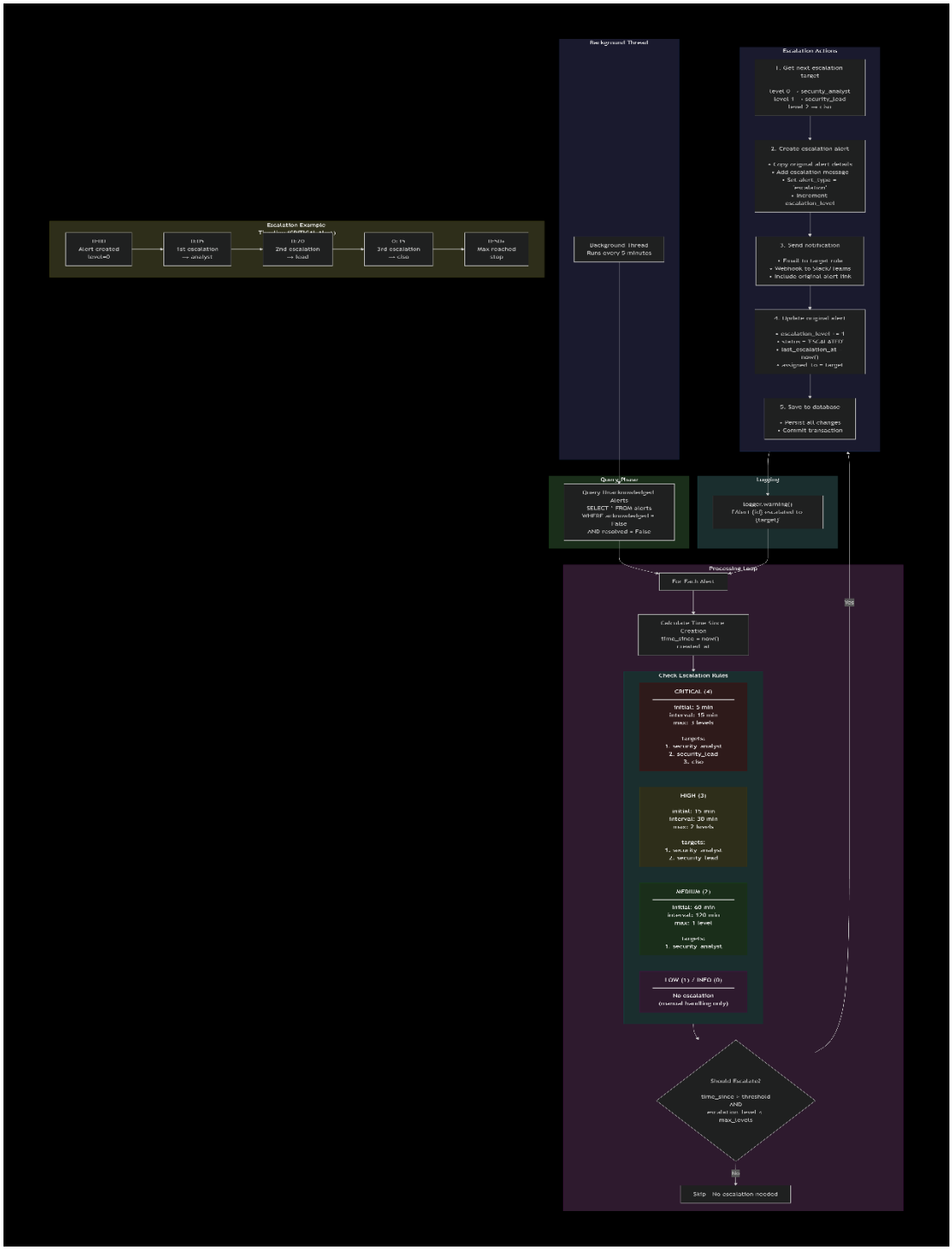
3.4.2 Crawl Queue Management



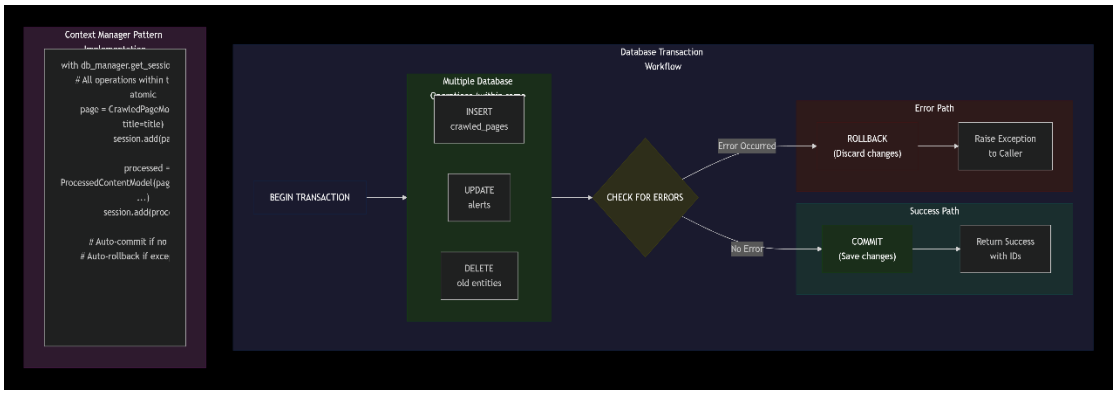
3.4.3 Keyword Detection Two-Pass Strategy



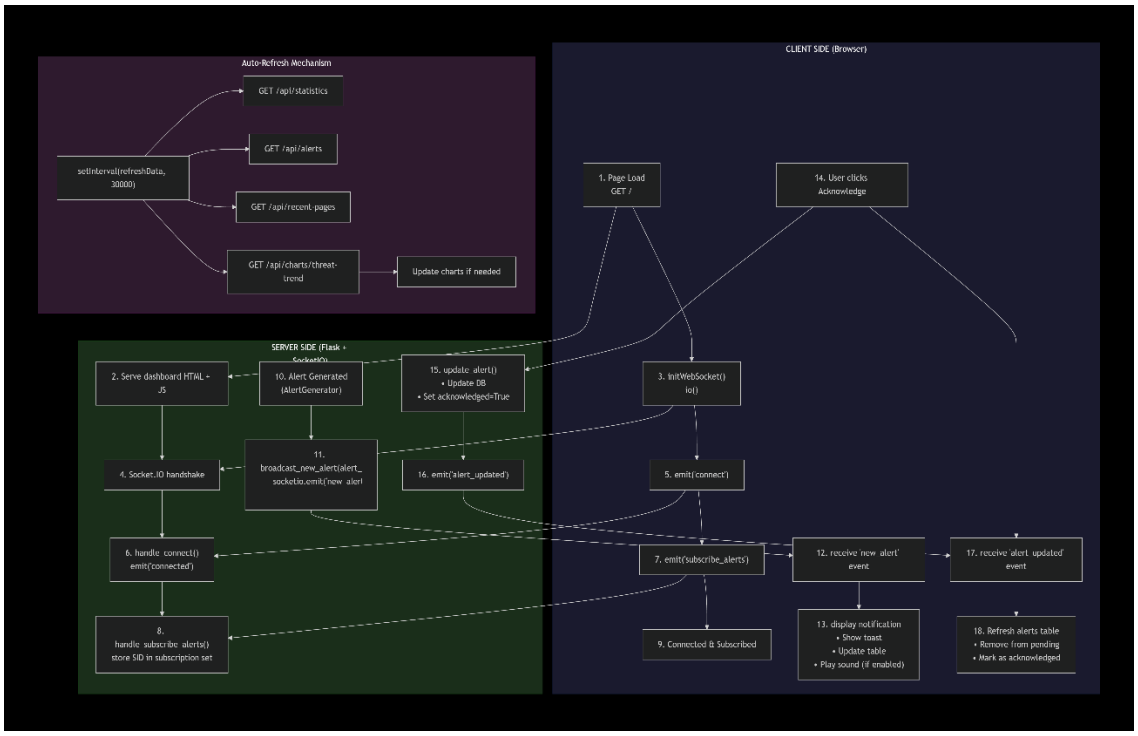
3.4.4 Alert Escalation



3.4.5 Database Transaction



3.4.6 Dashboard Real-Time Update



3.5 API Design

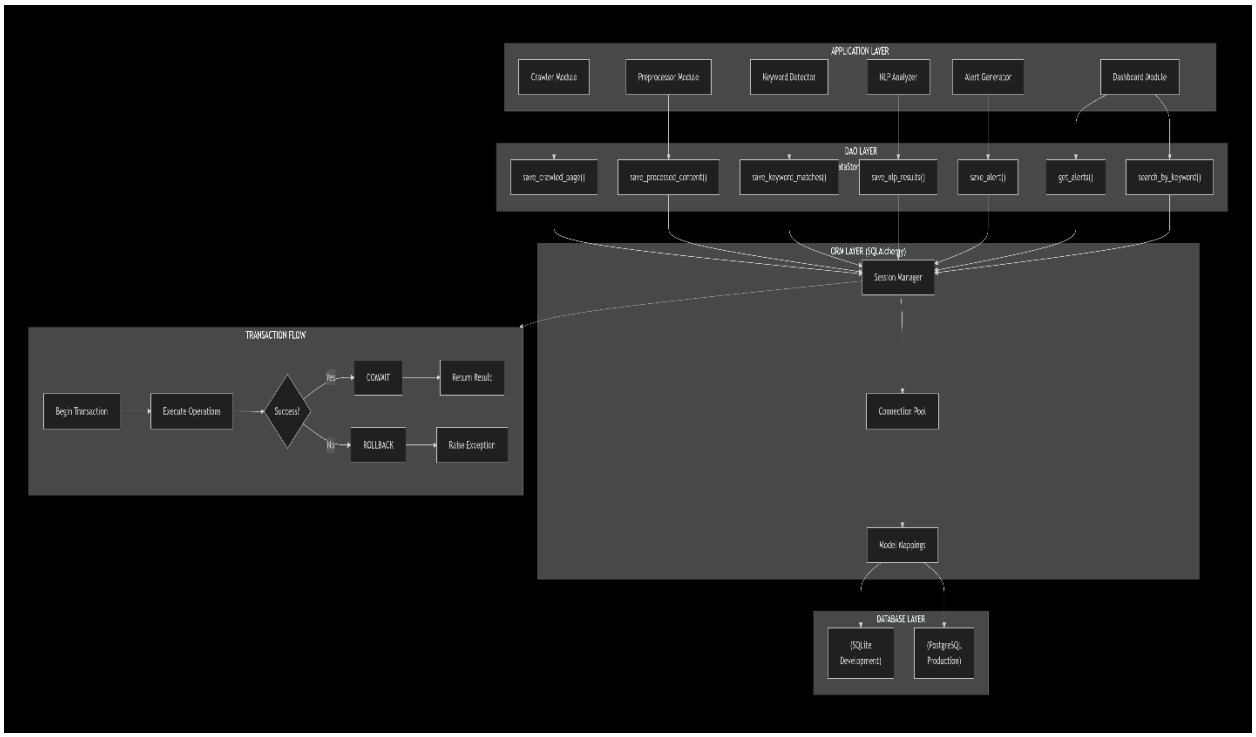
The Dark Web Monitoring System exposes a set of RESTful APIs to enable interaction between system components and external interfaces such as dashboards. These APIs support key functionalities including data retrieval, alert management, and system monitoring. The following table provides a high-level overview of the available API endpoints.

Endpoint	Method	Description
/api/statistics	GET	Retrieve system-wide monitoring statistics
/api/alerts	GET	List security alerts with filtering
/api/alerts/{id}	PUT	Update alert status
/api/search	GET	Search across alerts and data
/api/recent-pages	GET	Retrieve recently crawled pages
/api/charts/threat-trend	GET	Provide threat trend data for visualization
/threat-timeline	GET	Threat score timeline for charts
/charts/threat-trend	GET	Plotly chart data for threat trends
/charts/alert-distribution	GET	Plotly pie chart for alert distribution

3.6 Database Interaction Design

Layer	Component	Responsibility
Application	Crawler, Preprocessor, NLP, Alert, Dashboard Modules	Business logic and data processing
DAO Layer	DataStorageServie	High-level-data operation
ORM Layer	SQLAlchemy ORM	Object-relational mapping, session managemet
Database Layer	SQLite	Connection pooling, engine mangement

Connection Layer	Database Manager	Connection pooling, engine

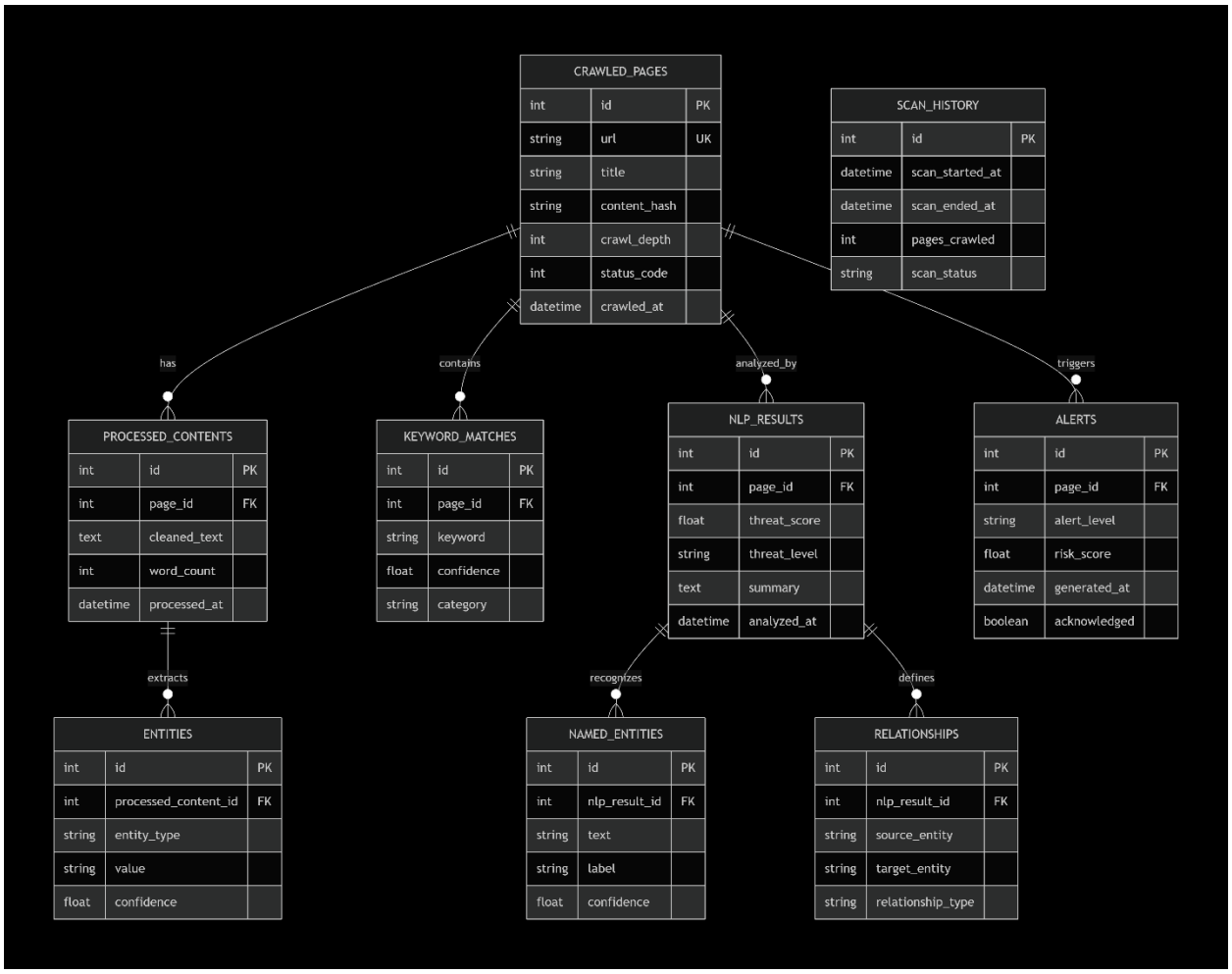


3.7 Data Structures & Schema Details

The database schema consists of 9 core tables that store everything from raw crawled pages to processed analysis results and security alerts. The design follows normalization principles with appropriate foreign key constraints and indexes for query performance. Each table is mapped to a SQLAlchemy ORM model, enabling type-safe database operations while maintaining flexibility for future schema changes through Alembic migrations. Sensitive data such as credentials are automatically redacted before storage, storing only partial values for identification without exposing complete secrets.

Table Name	**Purpose**	**Key Fields**	**Relationship**
Crawled_pages	Stored metadata for each crawled dark web page	Id(PK), url, content_hash, crawled_at	Parent to all analysis tables
Processed_contents	Cleaned and preprocessed	Id (PK), page_id (FK), cleaned_text,	Belongs to crawled_pages

	text content	word_count	
Entities	Extracted emails, domains, Ips, credentials	Id (PK), processed_content_id (FK), entity_type, value	Belongs to processed_contents
Keyword_matches	Keyword detection results	Id (PK), page_id(FK), keyword, confidence, category	Belongs to crawled_pages
Nlp_results	NLP analysis with threat sccores	Id (PK), page_id(FK), threat_score, summary	Belongs to crawled _pages
Scan_history	Audit trail of monitoring scan sessions	Id (PK), scan_started_at, scan_status	Standalone



3.8 State and Session Management

The system manages state across multiple layers to ensure efficient processing and persistence.

Layer	State Type	Storage	Scope
Crawler	Crawl progress	In-memory	Per crawl session
Tor	Network identity	In-memory	Application runtime
Database	Pages, alerts, entities	Persistent	Permanent
Dashboard	User Session	In-memory	Client Session
Alert	Alert lifecycle	Database	Permanent

- Crawler State Management

  The crawler maintains runtime state for efficient crawling.

  Key States:
  Visited URLs (to avoid duplicates)
  URL Queue (pending pages)
  Domain visit counts

  Characteristics:
  Stored in memory
  Lost on restart
  Ensure efficient crawling

- Database Session Management

  The system uses a session-based database access pattern.

  Features:
  Automatic commit on success
  Rollback on failure
  Connection pooling for performance

- Alert State Management

Key Fields

Status (new, escalated, resolved)
Acknowledged (true/false)
Escalation_level

- Dashboard Session Management

The dashboard maintains user session state.

Stored Data:
Session ID

Filter preferences
Active connections (WebScoket)

Characteristics:
Stored in memory
Lost on disconnect
Auto-reconnect supported

3.9 Caching Mechanism

Caching is used to temporarily store frequently accessed or computationally expensive data so that repeated processing can be avoided.

In the Dark Web Agriculture Monitor, caching helps to:

- Improve system performance by reducing redundant operations
- Minimize repeated crawling of identical content
- Reduce load on external systems such as the Tor network and NLP models
- Optimize database and network resource usage

Types of Caching Used

Cache Type	Purpose	Storage	TTL
Content Deduplication Cache	Avoid processing Duplicate pages	In-memory	Session

Alert Deduplication Cache	Prevent duplicate alerts	In-memory	60 minutes
NLP Model Cache	Reuse loaded NLP models	In-memory	Permanent
Regex Pattern Cache	Reuse compiled patterns	In-memory	Permanent
Data Connection Pool	Reuse DB connections	Memory	1 hours
Session Cache	Reuse HTTP connections	Memory	Session

Key Caching Mechanisms

1. Content Deduplication Cache

    - Stores hash values of recently crawled content
    - Prevent reprocessing identical or similar pages
    - Improves crawling efficiency

2. Alert Deduplication Cache

    - Stores recent alerts using a unique hash
    - Prevents duplicate alerts within a defined time window
    - Reduces alerts noise for analysts

3. NLP Model Cache

    - NLP models (spcaCy, Hugging Face) are loaded once and resused
    - Eliminates repeated model for loading time
    - Significantly improves processing speed

4. Regex Pattern Cache

    - Keyword patterns are compiled once at startup
    - Reused for all document processing
    - Reduces repeated computation overhead

5. Database Connections Pool Cache

- Maintain a pool of reusable database connections
- Avoid overhead of creating new connections for each request
- Improves query performance

6. Session Cache (HTTP Requests)

- Reuse HTTP sessions and connections
- Maintains headers and connection pooling
- Reduces latency for repeated request

Cache Invalidation Strategy

Cache Type	Invalidation Method
Content Cache	FIFO
Alert Cache	Time-based expiration
NLP Models	Non invalidation (static)
Regex Cache	Reload on config change
DB Connections	Recycled periodically
Session Cache	Reset on identity rotation

Benefits of Caching

- Fast data processing and response time
- Reduced redundant crawling and analysis
- Lower network and computational overhead
- Improved scalability of the system
- Better user experience in dashboard and alerts

3.10 Security Implementation

Security is a critical requirement for the Dark Web Agriculture Monitor, as the system interacts with potentially sensitive and high-risk environments. The system is designed to ensure user anonymity, protect collected data, and secure all communications with external systems.

- Anonymity and Privacy

  The system ensures complete anonymity while accessing dark web resources

Requirements	Implementation
IP Anonymity	All requests routed through Tor network.
Identity Rotation	Periodic IP rotation using Tor (NEWNYM signal)
DNS Leak Prevention	Remote DNS resolution via SOCKS5h
Safe Crawling	Read-only requests

- Data Security

Sensitive data collection from dark web sources is handled carefully to prevent exposure.

Data Type	Protection Mechanism
Credentials	Stored in redacted form
Database	Access controlled via authentication
Logs	Restricted access and periodic rotation
Configuration Files	Secured with limited file permissions

- Network Security

All external communication are secured to prevent interception and misuse.

Components	Security Measure
Tor Communication	Encrypted routing via Tor circuits
Email Alerts	SMTP with STARTTLS
Webhooks	HTTPS-based communication
API Access	Restricted endpoints

- Access Control

Access to system components is controlled to prevent unauthorized page

Dashboard access is currently limited to trusted users

Future implementations includes authentication mechanism such as JWT or OAuth2

- Security Risks and Mitigation

Risk	Mitigation
Exposure of system IP	Use of Tor network
Unauthorized dashboard access	Implement authentication
Data leakage	Redaction and controlled storage
API abuse	Rate limiting (future Enhancement

3.11 Performance Optimization

The performance requirements define how efficiently the Dark Web Agriculture Monitor processes data, responds to user requests, and utilizes system resources. These requirements ensure the system operates within acceptable time limits while handling continuous dark web monitoring workloads.

Performance Objective

The system is designed to achieve the following performance goals:

- Provide fast crawling and analysis of dark web pages
- Ensure low-latency dashboard responses for users
- Minimize processing delays in NLP and alert generation
- Optimize resource utilization (CPU, memory, network)
- Maintain consistent performance under moderate load

Key Performance Metrics

Metric	Target	Description
Page Crawl Time	10 seconds	Time to fetch a dark web page via Tor
Content Preprocessing	< 500 ms	Cleaning and parsing HTML/text
Keyword Detection	< 200 ms	Pattern matching execution time
NLP Analysis	< 5 sec	Entity extraction and classification
End-to-End Processing	< 15 sec	Full pipeline per page
API Response Time	< 200 ms	Dashboard API latency
Alert Generation	< 100 ms	Time to create alert after detection
DB Query Time	< 50 ms	Indexed query performance
WebScoket Delivery	< 500 ms	Real-time alert push delay

Throughput Capacity
The system support the following processing capacity:

Metric	Value
Page per Hour	~ 300 - 600
Page per Day	~ 7,000 – 14,000
Alerts per Day	~ 50 - 500

Resource Utilization

Resource	Usage	Notes
CPU	30-50%	Peak during NLP processing
Memory	2-3 GB	NLP models consume majority
Network	1-5 Mbps	Limited by Tor bandwidth
Disk	< 5GB	Depends on storage backend

Performance Optimization Techniques

1. Caching
    - NLP models loaded once (reduces repeated load time)
    - Regex patterns precompiled
    - Content deduplication avoids reprocessing
2. Connection Pooling
    - Database connection reuse (QueuePool)
    - HTTP session reuse for Tor requests
3. Content Deduplication
    - Hash-based detection avoids duplicate crawling
    - Reduces workload by ~15–25%
4. Batch Processing
    - Processes multiple pages in batches
    - Improves CPU efficiency

Scalability Consideration

Vertical Scaling

- Increase CPU/RAM for NLP workloads
- Use GPU acceleration for large models

Horizontal Scaling

- Multiple crawler instance
- Distributed caching using Redis
- Shared database

Scalable Components

- Crawlers (Stateless, easily replicated)
- Dashboard (load-balanced)
- Alert processing (queue-based)

Performance Constraints

The system performance is affected by:

- Tor Network Latency
- Dark Web Site Availability
- Large NLP Models
- Single-instance limitations

4. Interfaces

The system interfaces are categorized into four types:

- External Interfaces – Interaction with external systems (Tor, email, webhooks)
- Internal Interfaces – Communication between system modules
- User Interface – Dashboard for monitoring and analysis
- Configuration Interface – YAML-based configuration management

❖ External Interfaces

These interface enable communication with systems outside the application.

Tor Network Interface	• Provides anonymous communication using Tor • Uses SOCKS5h protocol via local Tor daemon • Support identity rotation for privacy
Dark Web Site Interface	• Fetch content from .onion websites • Uses HTTP/HTTPS requests • Includes retry and delay mechanisms
Email Interface	• Send alert notifications to users • Uses SMTP with secure connection • Support multiple recipients
Webhook Interface	• Sends alerts to platforms like Slack • Uses HTTP POST with JSON payload • Enable integration with external systems

❖ Internal Interfaces

Internal interfaces define communication between system modules

Caller	**Callee**	**Purpose**
Main	Crawler	Start crawling process
Crawler	Preprocessor	Clean extracted content
Preprocessor	Keyword Detector	Detect sensitive keywords
Keyword Detector	NLP Analyzer	Analyze context and intent
NLP Analyzer	Alert Generator	Generate alerts
Alert Generator	Storage	Save alerts
Dashboard	Storage	Fetch alerts and statistics

❖ Data Transfer Objects

Data Transfer Objects are used to transfer structured data between modules

Data Transfer Objects	**Purpose**
CrawledPage	Stores raw page data
ProcessedDocument	Stores cleaned content
DocumentMatchResult	Stores keyword detection results
NLPResult	Stores analysis output
Alert	Stores alert details

- User Interface

Dashboard

The system provides a web-based dashboard for monitoring

Features:

→ View alerts and threat levels
→ Search and filter data
→ View statistics and trends
→ Acknowledge alerts

API Endpoints

Endpoint	**Purpose**
/api/statistics	Get system metrics
/api/alerts	Fetch alerts
/api/charts	Get visualization data
/api/recent-pages	View recent crawled pages

- Configuration Interface

The system uses YAML files for configuration

Key Configuration Files

File	**Purpose**
Settings.yaml	General system settings
Keywords.yaml	Keywords to monitor
tor_config.yaml	Tor network settings
alerts.yaml	Alert rules and threshold

5. Testing Strategy

The testing strategy for the Dark Web Agriculture Monitor follows a pyramid approach, emphasizing fast unit tests at the base, integration tests in the middle, and a small number of end-to-end tests at the top. This ensures comprehensive coverage while maintaining fast execution times and easy debugging.

Key Test Scenarios

Test ID	Category	Description	Success Criteria
T-01	Unit	Config loads YAML	All settings accessible via dot notation

		correctly	
T-02	Unit	URL normalization removes trailing slashes	http://site.onion/page/ → http://site.onion/page
T-03	Unit	Credential redaction masks sensitive data	Full password never stored or logged
T-04	Integration	Tor → Crawler flow	Page fetched successfully via Tor
T-05	Integration	Full pipeline (crawl to alert)	Alert generated for test keyword
T-06	Performance	100 pages crawl throughput	Average < 10 seconds per page
T-07	Security	Tor anonymity verification	IP masked, Tor detected

6. References

1. Chen, H., & Wang, F. (2021). "Dark Web Monitoring for Critical Infrastructure Protection." *Journal of Cybersecurity Research*, 12(3), 45-67.
2. Kumar, A., & Singh, R. (2022). "NLP-based Threat Detection on Dark Web Platforms." *International Conference on Cyber Security*, 112-128.
3. United Nations FAO. (2022). *Digital Agriculture and Cybersecurity Guidelines*. https://www.fao.org/
4. IBM Security. (2023). *Agricultural Sector Cybersecurity Threat Intelligence Report*.